授業受講者以外への再配布禁止です
Redistribution to anyone who don't take this lecture is prohibited

# 航空宇宙設計最適化

## Design optimization in aerospace engineering

大山　聖
Akira Oyama

# スケジュール　Schedule

| | | |
|---|---|---|
| 4/16 | [1] 設計最適化問題とは？ | What is design optimization? |
| 4/23 | [2] 設計最適化手法（その１） | Design optimization methods (part 1) |
| 5/7 | [3] 設計最適化手法（その２） | Design optimization methods (part 2) |
| 5/14 | [4]設計最適化手法（その３） | Design optimization methods (part 3) |
| 5/21 | [5]多目的設計探査 | Multiobjective design exploration |
| 5/28 | 休講 | |
| 6/4 | [6]飛行機の設計 | aircraft design |
| | JAXA第三研究ユニット長 嶋英志氏 | |
| 6/11 | [7]【活用事例】Solar-Dミッションの軌道計画 | |
| | | Trajectory design of Solar-D mission |
| 6/18 | [8]【活用事例】H-III ロケットのターボポンプ設計 | |
| | | H-III Rocket turbopump design |
| 6/25 | [9] ロバスト設計・不確定性定量評価 | robust design /uq |
| | 東北大学流体科学研究所准教授　下山幸治氏 | |
| 7/2 | [10]【活用事例】火星飛行機＆自動車設計 | |
| | | Mars airplane and car structure design |
| 7/9 | [11] 超音速機設計 | supersonic aircraft design |
| | JAXA航空プログラムグループ超音速機チーム牧野好和氏 | |

# 前回の復習

previous lecture review

# 最適化問題 Design optimization problems

- ある設計条件において，与えられた制約条件のもと，目的関数を最小化（あるいは最大化）する変数値を見つける問題

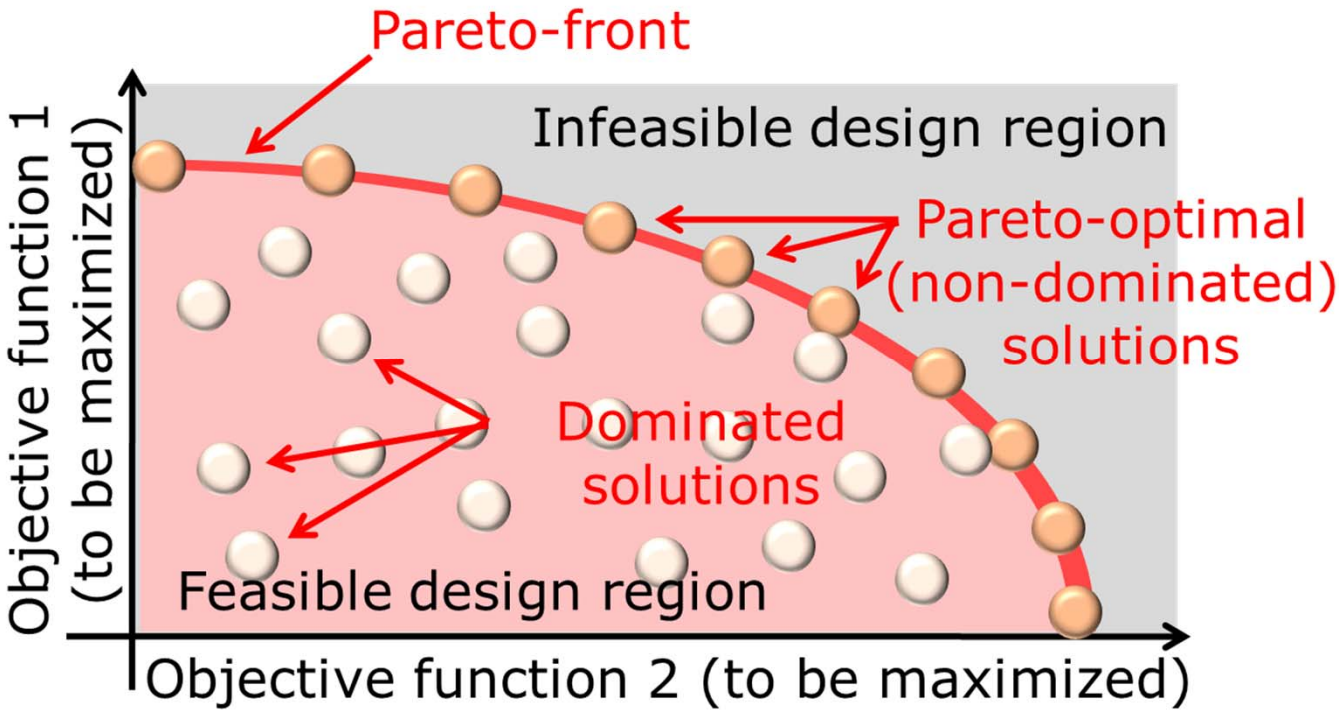
  Problem to find design parameter values that minimize (or maximize) objective functions in certain design conditions and under the constraints given

- 現実の設計問題のほとんどは多目的（複合領域）最適化問題

  Real-world design problems are usually multiobjective (multidisciplinary) optimization

- 多目的最適化問題の最適解は多数有り，パレート最適解と呼ばれる

  A multiobjective optimization problem has multiple optimal solutions called Pareto-optimal solutions



# 設計最適化問題の分類 Category of design optimization problems

| | 目的関数が1つ<br>One objective function | 目的関数が複数<br>Multiple objective function |
|---|---|---|
| 目的関数評価が1分野<br>Objective function in one field | 単目的設計最適化問題<br>Single objective design optimization | 多目的設計最適化問題<br>Multiobjective design optimization |
| 目的関数評価が複数分野<br>Objective function in multiple field | 複合領域設計最適化問題<br>Multidisciplinary design optimization | 多目的複合領域(多分野)設計最適化問題<br>Multiobjective multidisciplinary design optimization |

# 今日のアウトライン outline

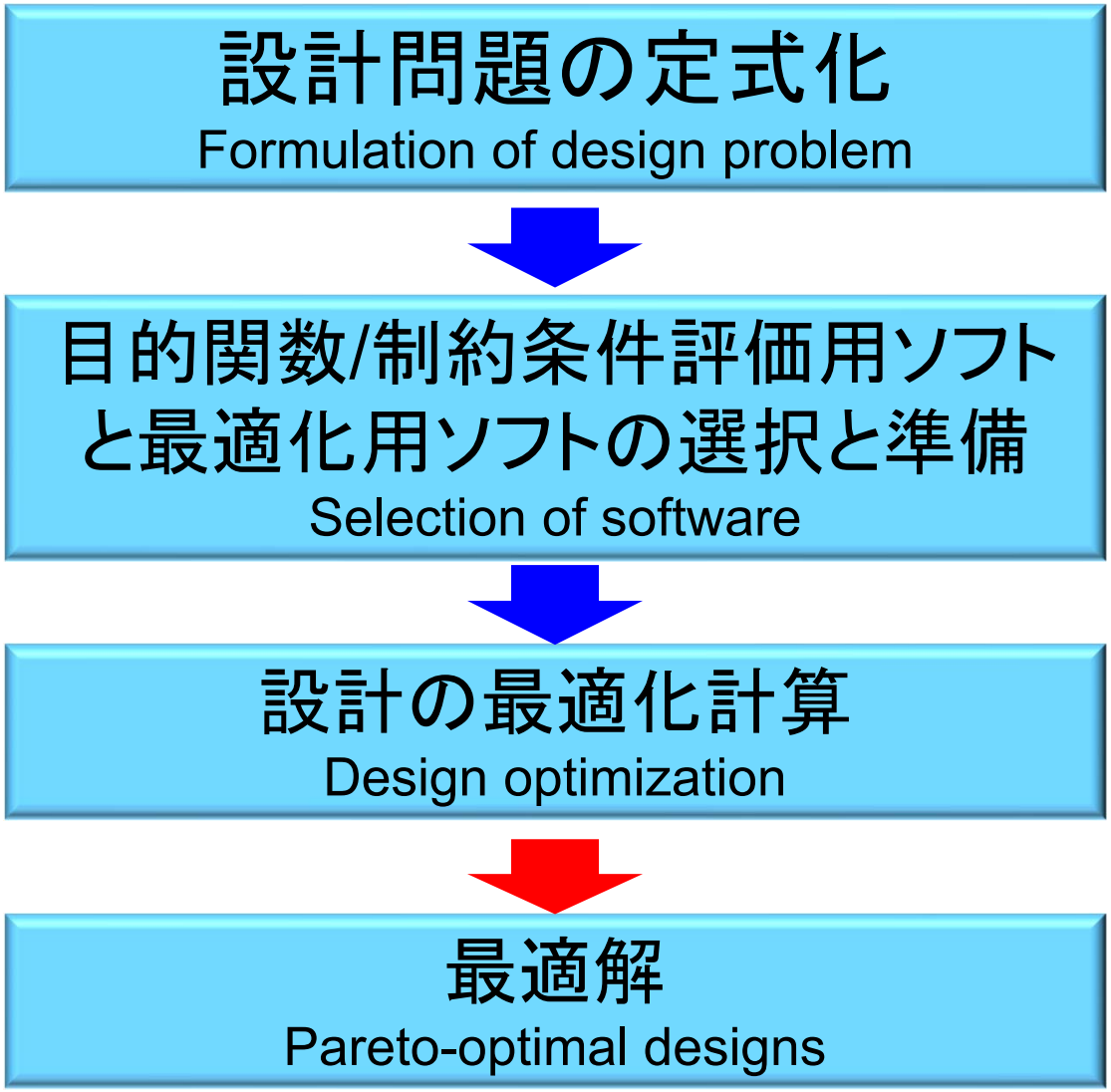
設計最適化の手順
Process of design optimization

設計最適化手法(1)
Design optimization methods(1)

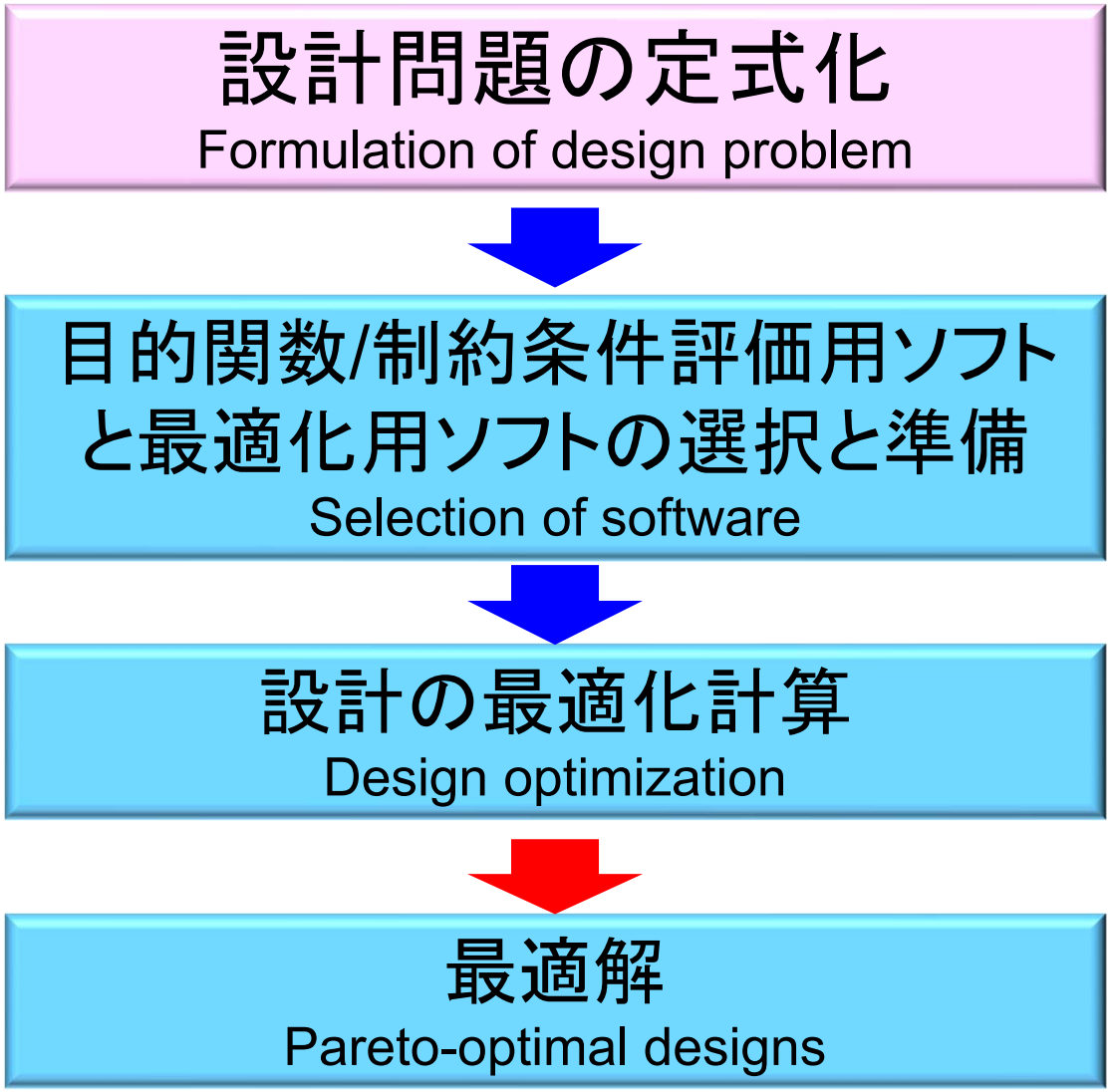
# 設計最適化の手順

Process of design optimization

# Flowchart


設計問題の定式化
Formulation of design problem
目的関数/制約条件評価用ソフト
と最適化用ソフトの選択と準備
Selection of software
設計の最適化計算
Design optimization
最適解
Pareto-optimal designs

# Flowchart


設計問題の定式化
Formulation of design problem
目的関数/制約条件評価用ソフト
と最適化用ソフトの選択と準備
Selection of software
設計の最適化計算
Design optimization
最適解
Pareto-optimal designs

# 翼型の空力設計最適化を考えます

we think about aerodynamic airfoil design



# 設計問題の定式化 Formulation of design problem

**設計最適化問題**
design optimization problem

ある環境下において，与えられた制約条件のもと，
目的関数を最小化（最大化）する変数を見つける問題
Problem to find design parameter values that minimize (or maximize) objective functions in certain design conditions and under the constraints given

定めなければならないこと to be defined

- 設計条件の定義 design conditions
- 目的関数の定義 objective functions
- 制約条件の定義 constraints
- 設計変数の定義 design parameters
- 探査領域の定義 design space

# 設計問題の定式化 –設計条件の定義-

## Formulation of design problem – design condition -

旅客機主翼の空力性能を向上させたい

To improve aerodynamic performance of an wing

そのために，主翼に利用する翼型の空力性能を向上させたい

To improve performance of a wing profile (airfoil)

- 設計条件はなんですか？

  What's the design condition?

  - 巡航速度 cruise speed　250m/s
  - 代表コード長　length　2m
  - 迎角 angle of attack　2deg.



www.mhi.co.jp

# 設計問題の定式化 –目的関数と目的の定義-

Formulation of design problem – design objectives -

空力性能を向上させるとはどういうことか？

What is “improvement of aerodynamic performance?”



翼型の揚力を最大化したい，抗力を最小化したい

Maximization of lift and minimization of drag



- 目的関数は？ Objective functions?
  - 揚力 lift
  - 抗力 drag
- 目的は？ Design objective?
  - 揚力の最大化 maximization of lift
  - 抗力の最小化 minimization of drag

# 設計問題の定式化 –制約条件の定義-

Formulation of design problem – constraints -

- 制約条件は？　　What are constraints?

  - 最大翼厚＞コード長の１２％

    Maximum thickness > 12% of chord length

    翼の厚みが薄いと揚力による曲げモーメントに耐えられないので最低限の翼厚は確保したい．．．

  - 揚力係数＞０

    Lift > 0

    負の揚力を持つ翼型は役に立たないので必要ない．．．

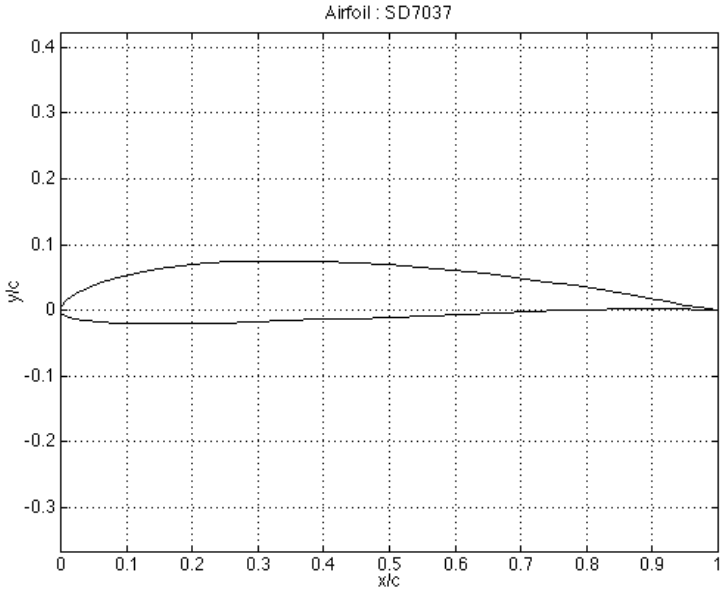
# 設計問題の定式化 –設計変数の定義-

Formulation of design problem – design parameters-

- 設計変数は？　　What are design parameters?

*翼型を設計するときはどうする？*

*How to parameterize an airfoil shape?*



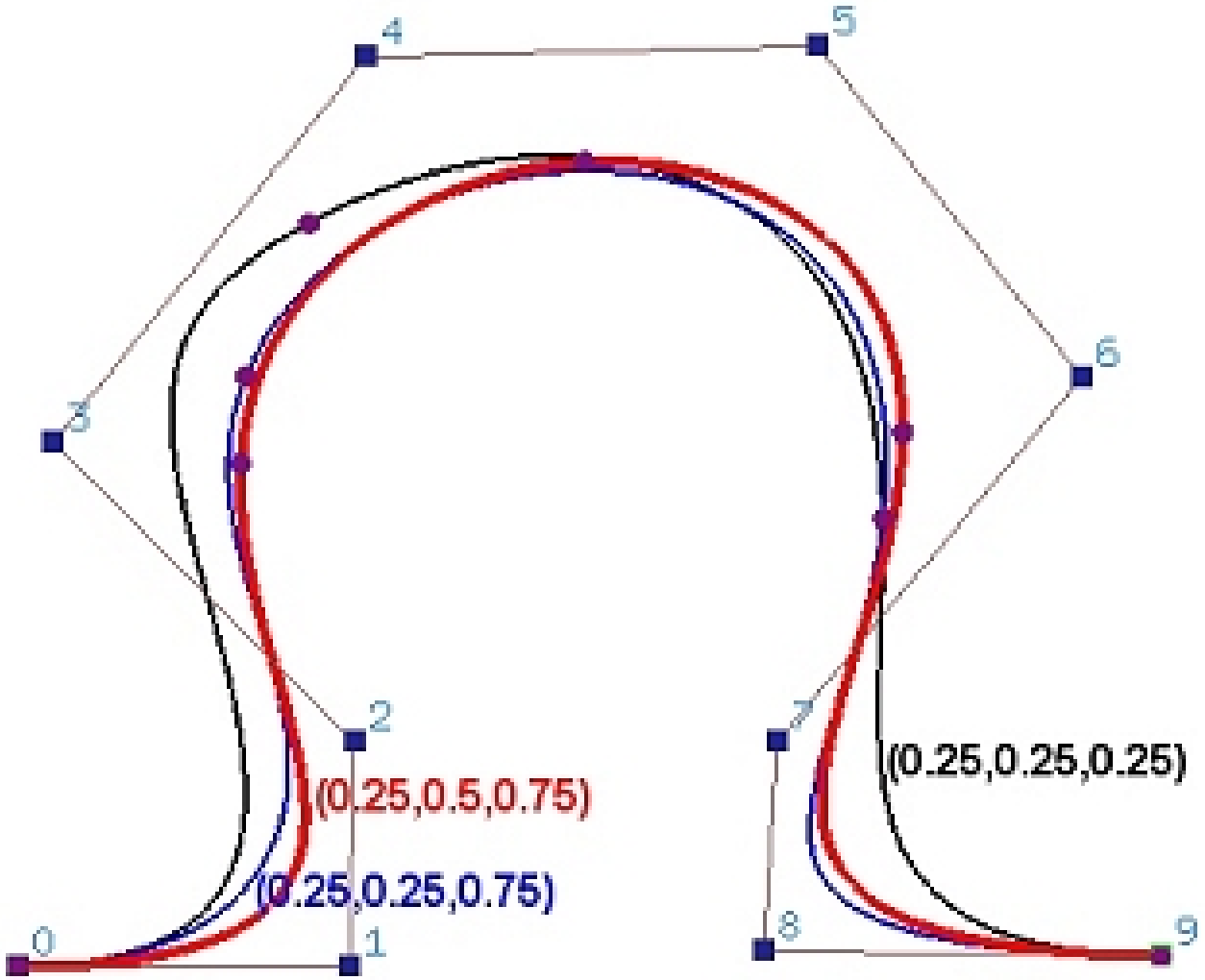



# 設計問題の定式化 –設計変数の定義-

## Formulation of design problem – design parameters-

- Bスプライン曲線を使って，翼型を定義する

  Parameterization with B-spline curves



# 設計問題の定式化 –設計変数の定義-

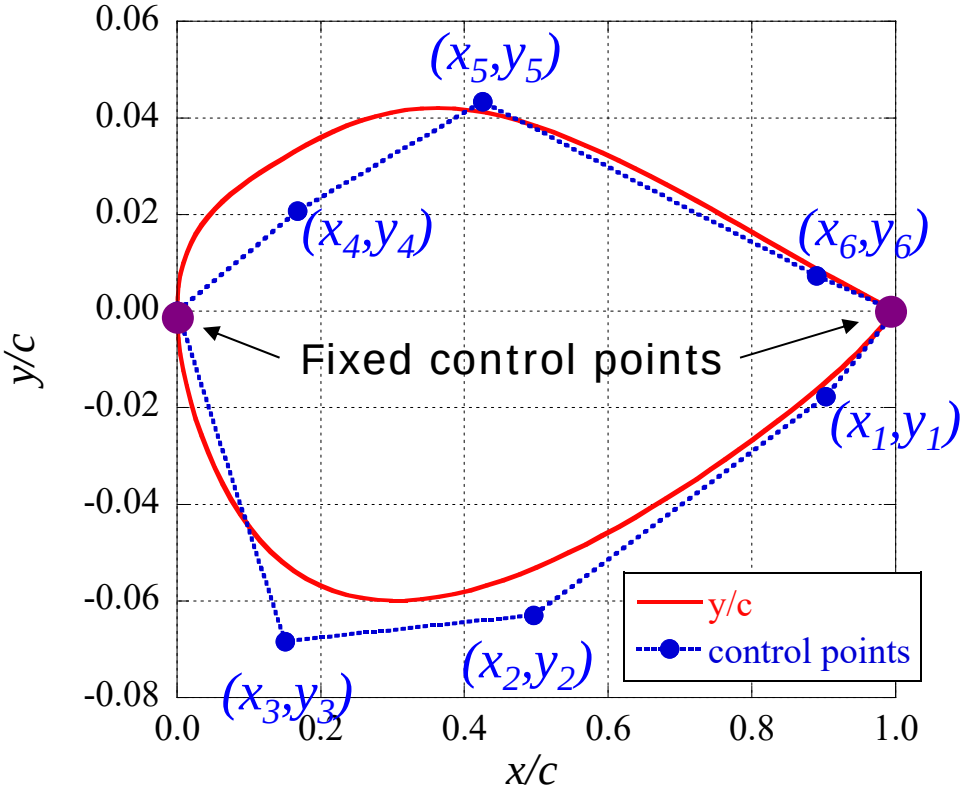
## Formulation of design problem – design parameters-

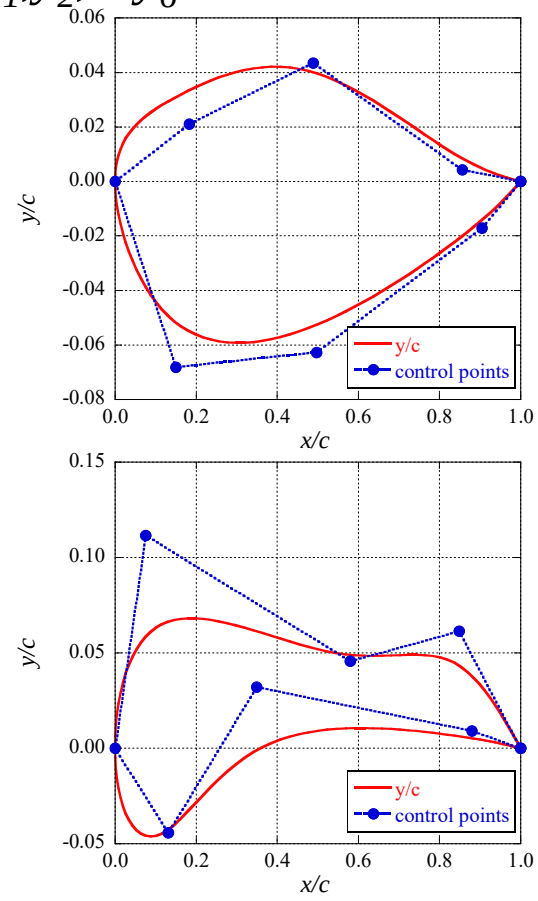
- 設計変数は6つの制御点のx座標及びy座標とする

  Design parameters are x and y coordinates of 6 control points

- 設計変数は $x_1,x_2,\ldots,x_6,y_1,y_2,\ldots y_6$ の１２個である

  Number of design parameters is 12: $x_1,x_2,\ldots,x_6,y_1,y_2,\ldots y_6$

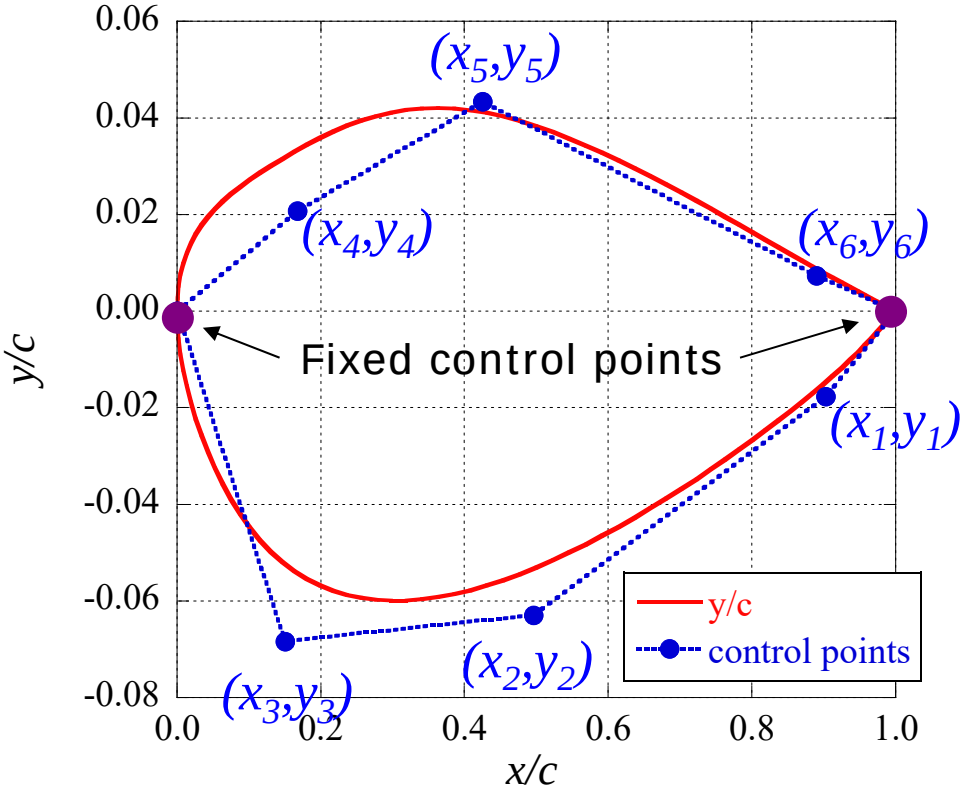


# 設計問題の定式化 –設計領域の定義-

## Formulation of design problem – design space-

- 広くとりすぎると最適解を探すのが難しくなる
  Larger, harder to find the optimal solutions
- 狭すぎると最適解が設計領域内に存在しない可能性がある
  Too small, possibility of excluding the optimal solutions




Design space

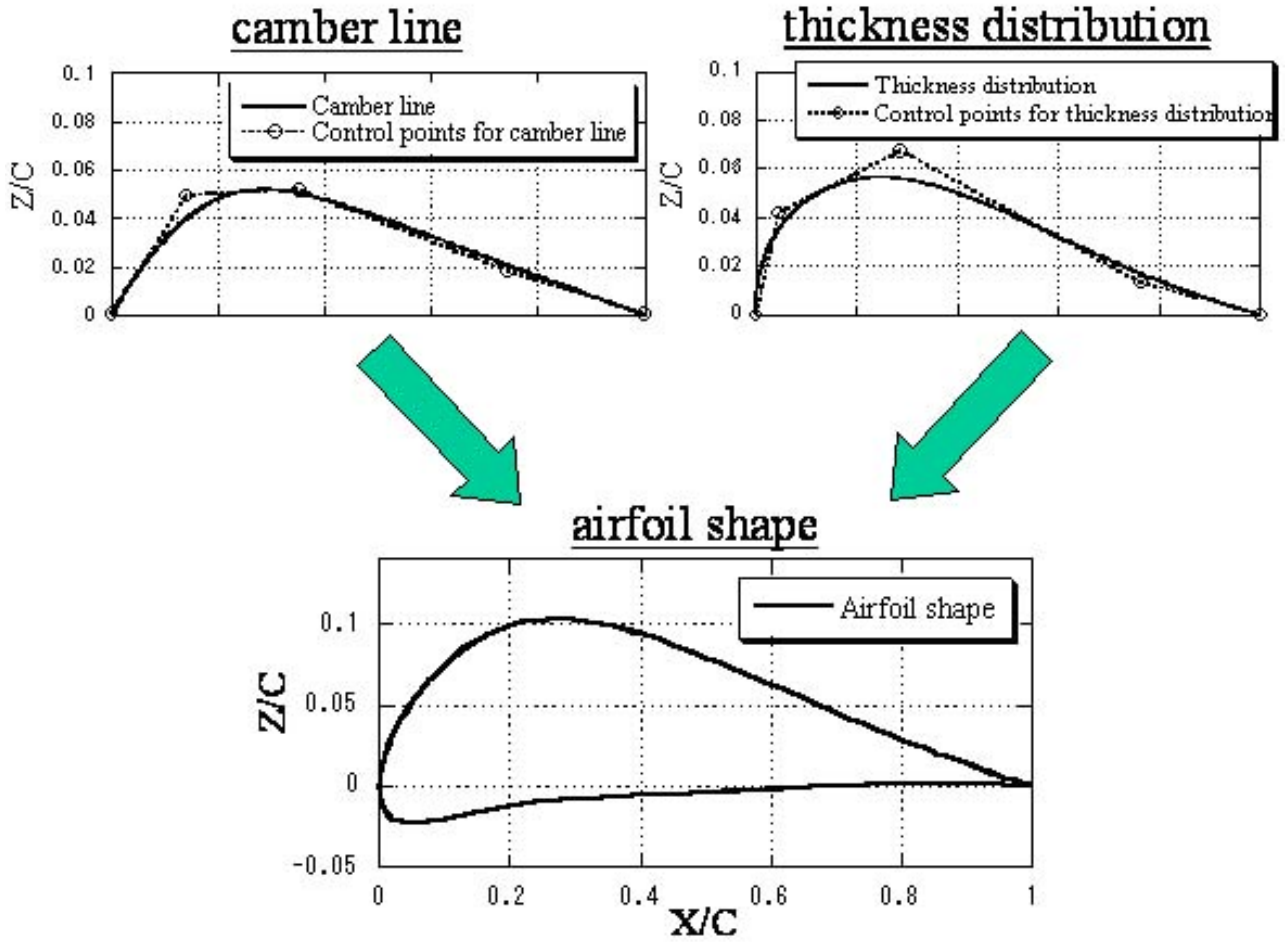
| | |
|---|---|
| *0.66<x1<0.99* | *-0.1<y1<0.1* |
| *0.33<x1<0.66* | *-0.1<y2<0.1* |
| *0.01<x3<0.33* | *-0.1<y3<0.1* |
| *0.01<x4<0.33* | *0.0<y4<0.2* |
| *0.33<x5<0.66* | *0.0<y5<0.2* |
| *0.66<x6<0.99* | *0.0<y6<0.2* |

# 参考資料:翼型の定義方法について

other parameterization of airfoil shape

## キャンバーと翼厚分布をBスプライン曲線で表現

Camber line and thickness distribution are parameterized with B-spline curves



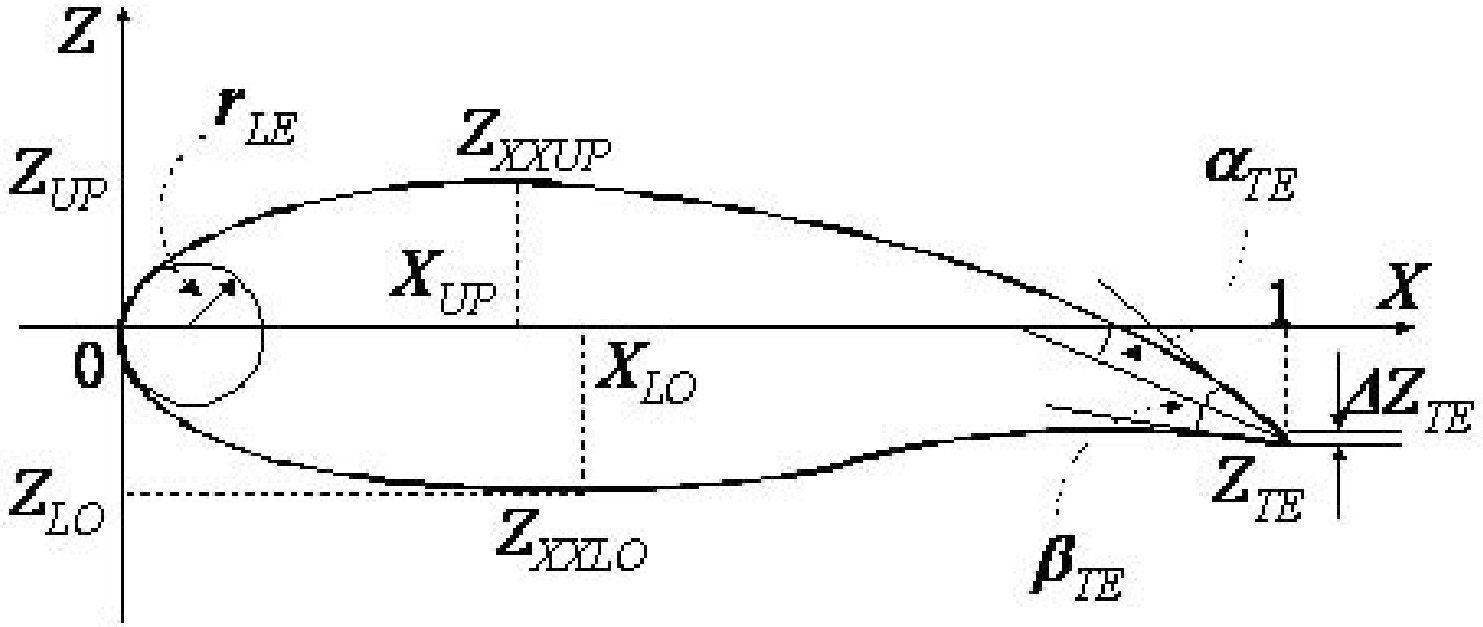

設計変数の数は任意 Number of design parameters is arbitrary

# 参考資料:翼型の定義方法について

other parameterization of airfoil shape

## PARSEC Airfoil

$$Z = \sum_{n=1}^{6} a_n X^{n-1/2}$$




$a_n$と設計変数との関係は解析的に求めることが可能

上下面でそれぞれ多項式を定義するので設計変数は12個

# 参考資料:翼型の定義方法について

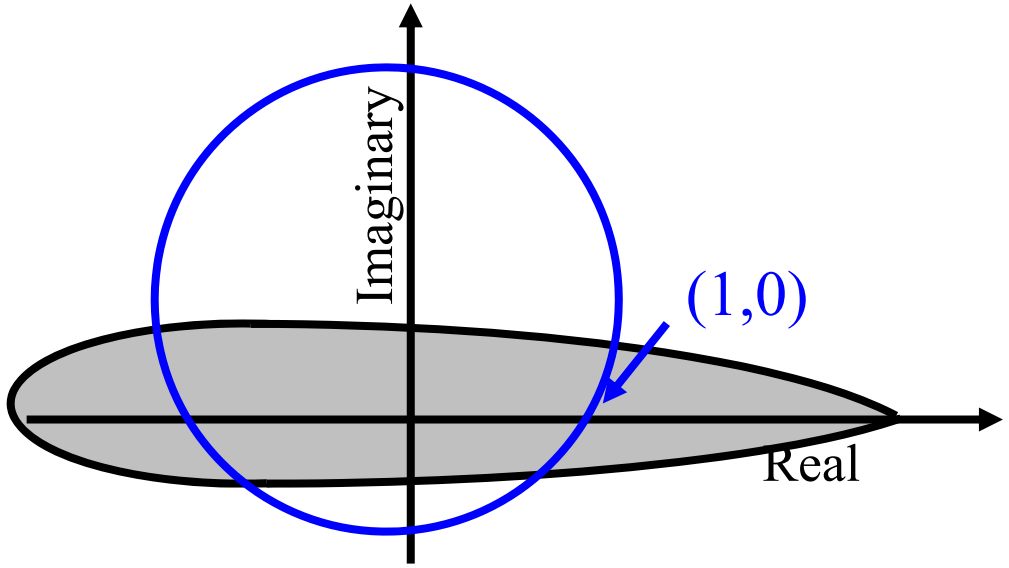
## other parameterization of airfoil shape

### Joukowsky Airfoil

(1,0)を通る円をジューコフスキー変換し翼型形状を得る
設計変数は変換される円の中心座標$(x_c,y_c)$のふたつ

Airfoil shape is obtained by Joukowsky transform of a circle.
Design parameters are coordinates of the center of the circle.

$$\varsigma = z + \frac{1}{z}$$



# 参考資料：翼型の定義方法について

other parameterization of airfoil shape

## Extended Joukowsky airfoil

(1,0)を通る円を拡張ジューコフスキー変換し，翼型形状を得る
設計変数は変換される円の中心座標
$(x_c,y_c)$, $\Delta$, $Real(\varepsilon)$, $Img(\varepsilon)$の5つ

Airfoil shape is obtained by two-stage transform of a circle. Design parameters are $(x_c,y_c)$, $\Delta$, $Real(\varepsilon)$, $Img(\varepsilon)$

$$\varsigma = z' + \frac{1}{z'}$$

$$z' = z - \frac{\varepsilon}{(z-\Delta)}$$

## Inverse Theodorsen's Transformation airfoil

単位円（中心(0,0)の半径1の円）
をセオドルセン変換の逆変換により
翼型形状を得る
設計変数の数は任意

Airfoil shape is obtained by Theodorsen's transform of a circle. Number of design parameters is arbitrary

$$\varsigma = z' + \frac{1}{z'}$$

$$z' = z \exp\left(\sum_{0}^{\infty} \frac{C_n}{z^n}\right)$$

# 翼型のパラメータ化手法はどれがよい？ Which one is the best?

Bスプラインでパラメータ化（表面形状）
B-spline approximation of airfoil surface

Bスプラインでパラメータ化（翼弦に翼厚分布を重ねる）
B-spline approximation of camber line and thickness distribution

PARSEC airfoil

Joukowsky Airfoil

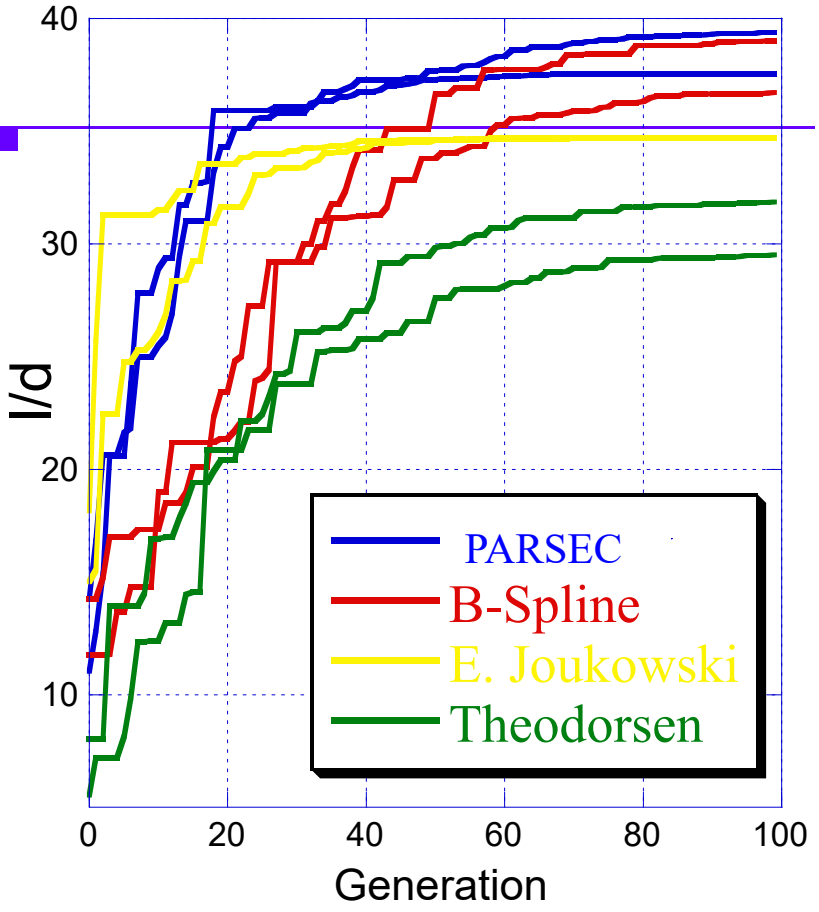
Extended Joukowsky airfoil

Inverse Theodorsen’s Transformation airfoil

# 実際に空力最適化をやってみると．．．

- 目的： 揚抗比の最大化
  Maximization of lift-to-drag ratio of an airfoil

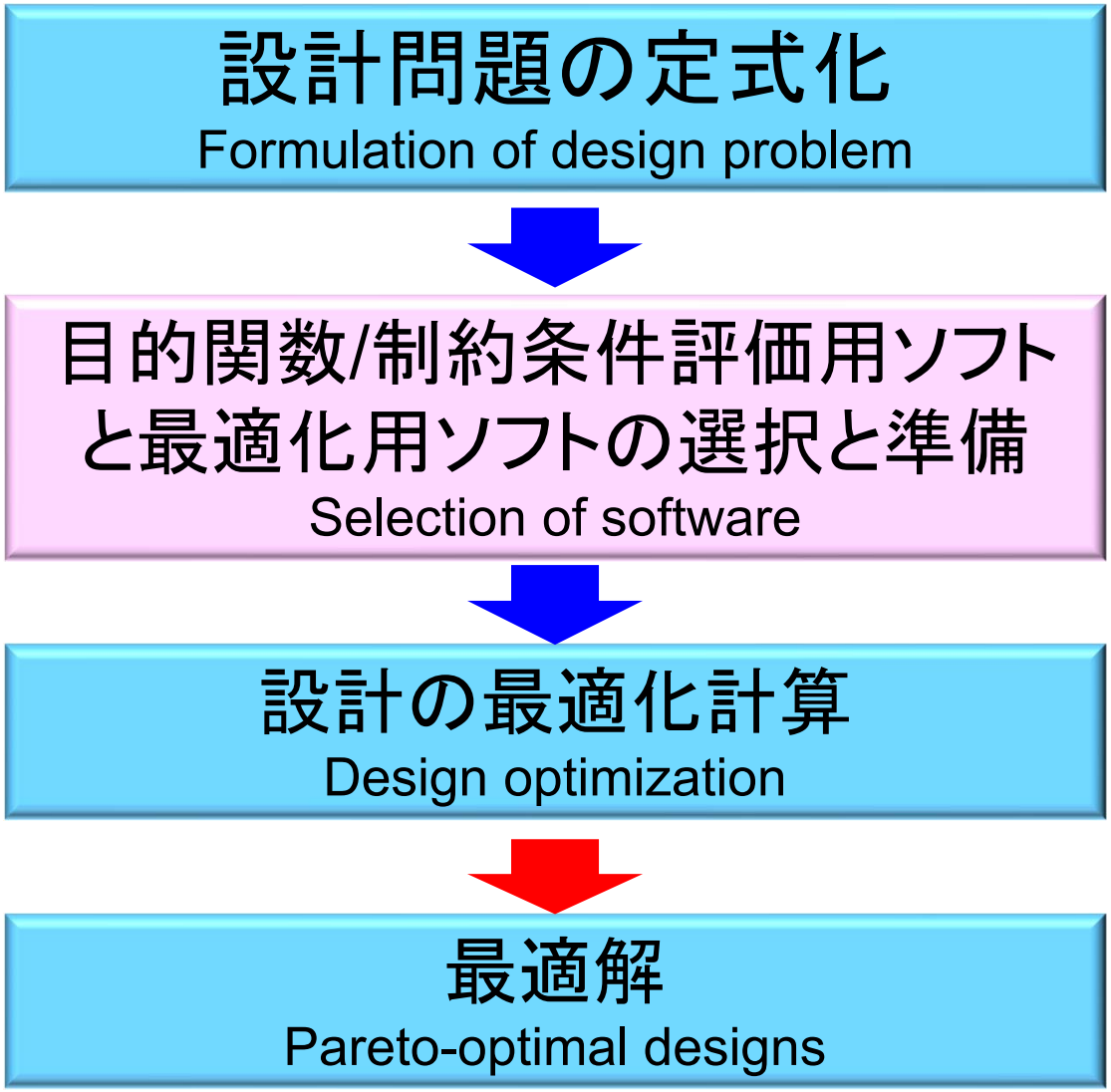
設計変数の取り方によって得られる最適解が異なる
Result depends on parameterization




**設計変数の取り方は大事**
Selection of parameterization is important

- パラメータは少ないほうがよい small number of parameters
- 探査領域を指定しやすい変数がよい easy to define design space
- 十分な設計空間をもつことが必要 sufficient search space
- 目的関数に直接的に関係したパラメータのほうがよい parameters directly related to objective function

# Flowchart


設計問題の定式化
Formulation of design problem
目的関数/制約条件評価用ソフト
と最適化用ソフトの選択と準備
Selection of software
設計の最適化計算
Design optimization
最適解
Pareto-optimal designs

# 目的関数/制約条件評価用ソフトと最適化用ソフトの選択と準備
## Selection of software

**要求にあった評価手法･最適化手法を選択すること！**
Choose software that are required!

### ■目的関数評価手法
Selection of objective function evaluation method

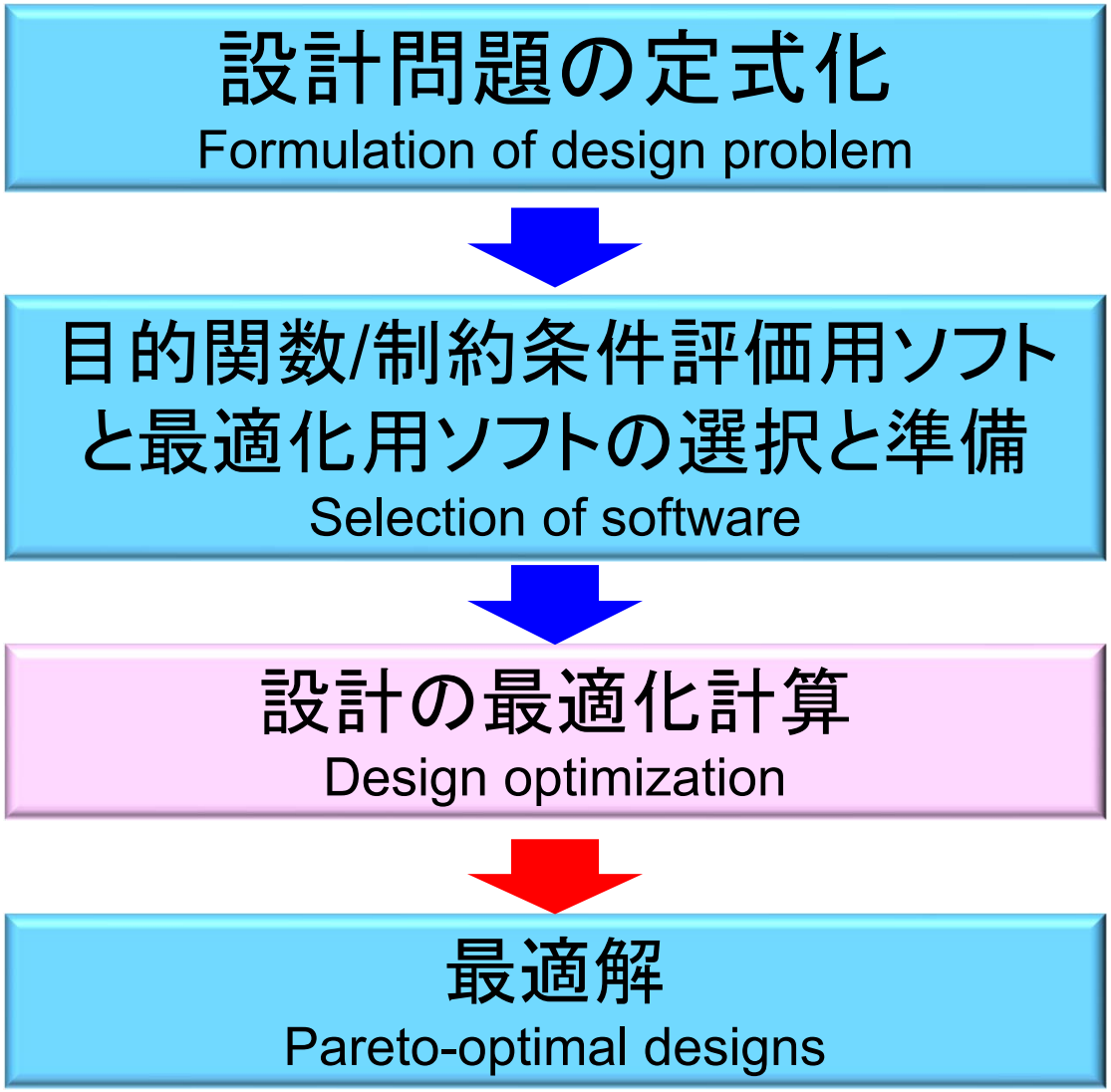
例）遷音速翼の揚力･抗力の評価手法
Example) evaluation of lift and drag of transonic wing

- 要求 Demand
  - 衝撃波の影響も考慮したい，粘性の影響も考慮したい．．．
    Effects of shock wave and viscosity should be considered
- 選択 Choice
  - ナヴィエストークス計算　Navier-Stokes solver

### ■設計の最適化手法 Selection of Design optimization method

このあと説明します It will be explained later

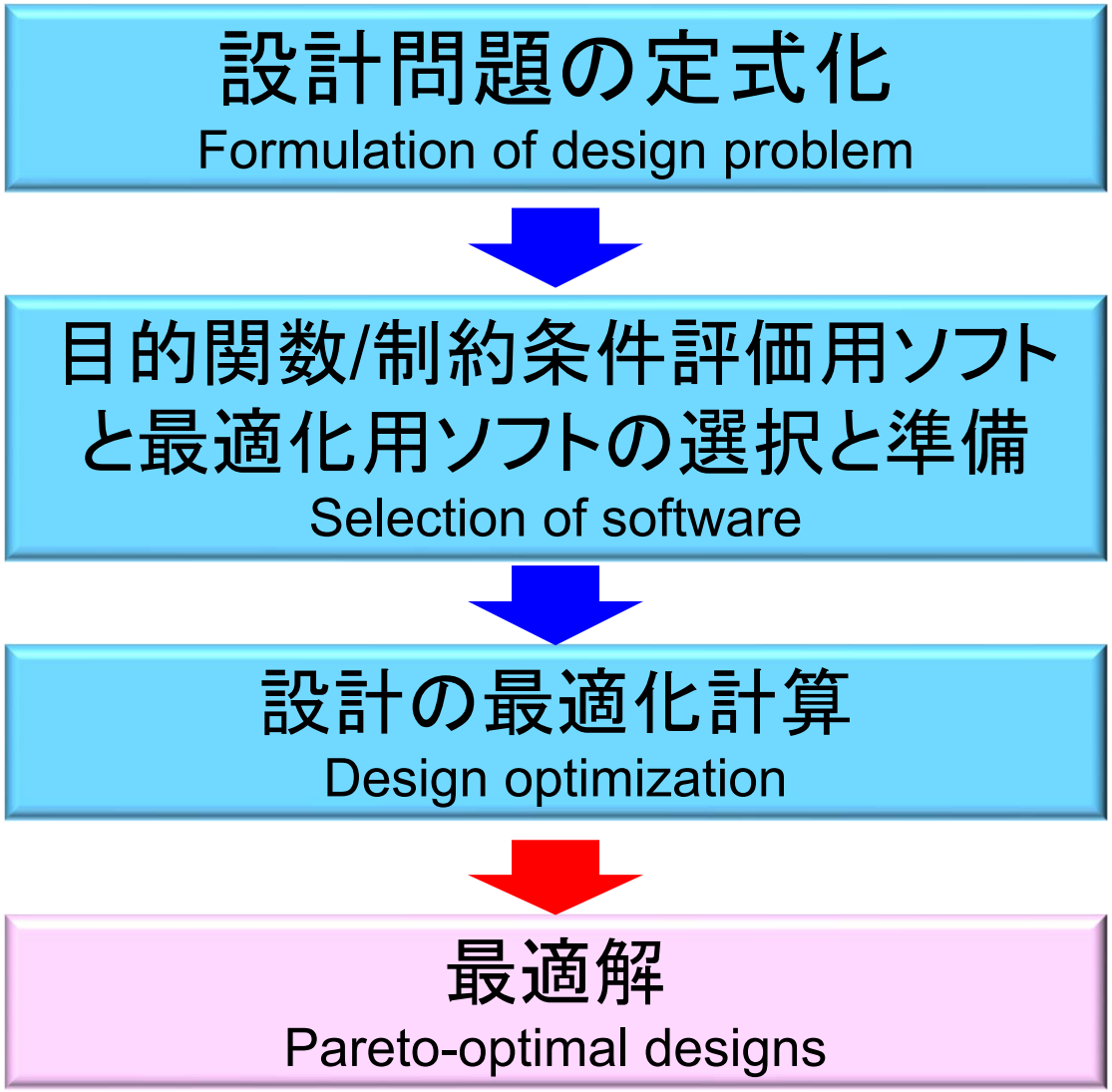
# Flowchart


設計問題の定式化
Formulation of design problem
目的関数/制約条件評価用ソフト
と最適化用ソフトの選択と準備
Selection of software
設計の最適化計算
Design optimization
最適解
Pareto-optimal designs

# 設計の最適化計算については
# このあとやります

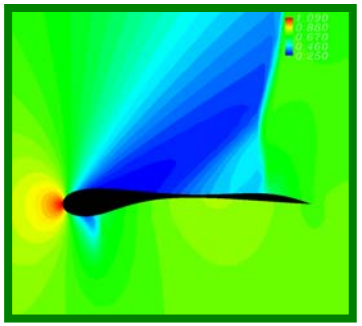
*Design optimization methods will be explained later*

# Flowchart

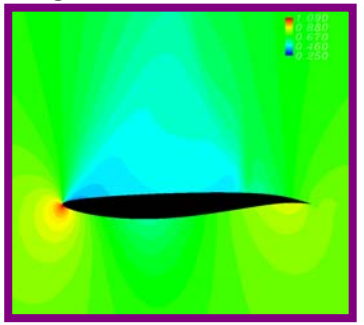

設計問題の定式化
Formulation of design problem
目的関数/制約条件評価用ソフト
と最適化用ソフトの選択と準備
Selection of software
設計の最適化計算
Design optimization
最適解
Pareto-optimal designs

# 得られる最適解の例 Pareto-optimal designs

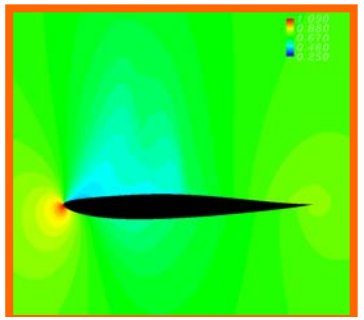
揚力係数最大の解（圧力分布）
Lift maximum design

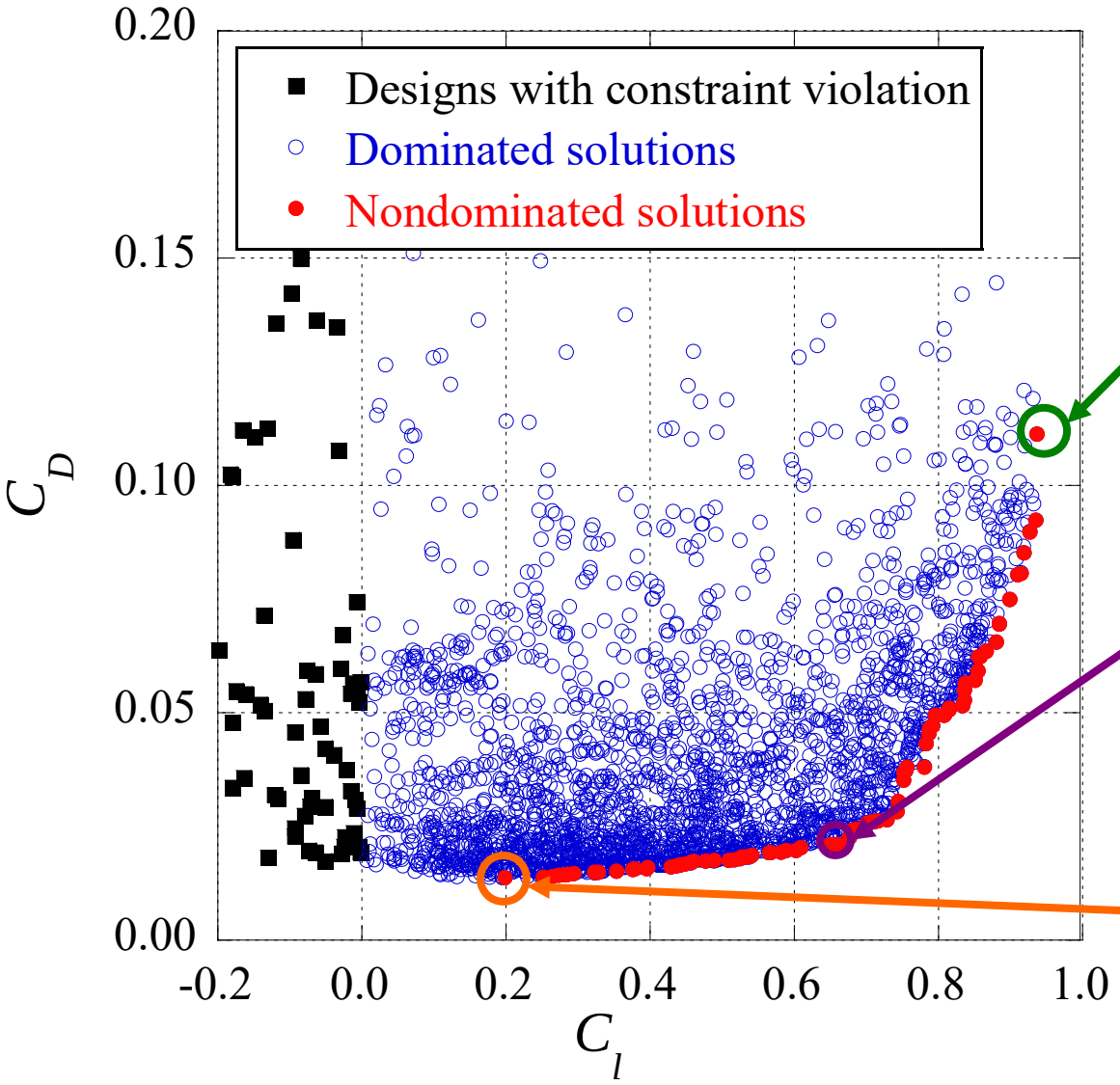


揚抗比最大の解（圧力分布）
Lift-to-drag ratio maximum design



抵抗係数最小の解（圧力分布）
Drag minimum design



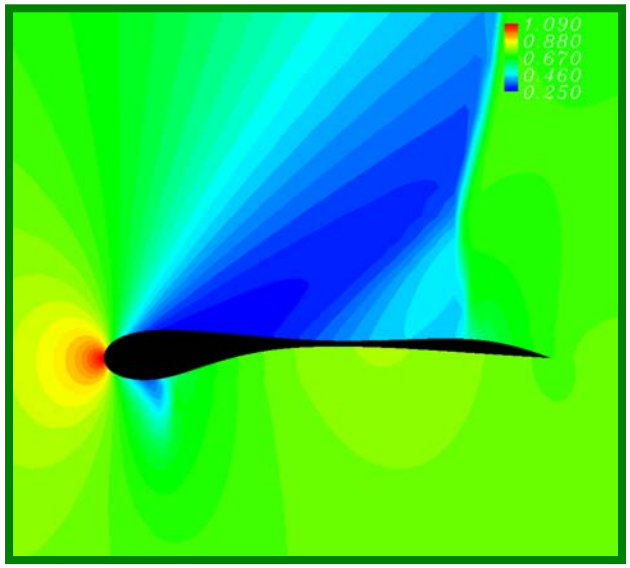


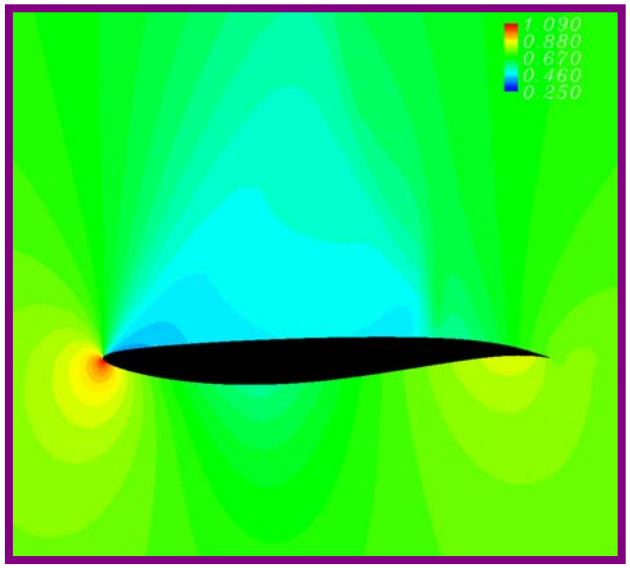
# 得られる最適解の例 Pareto-optimal designs

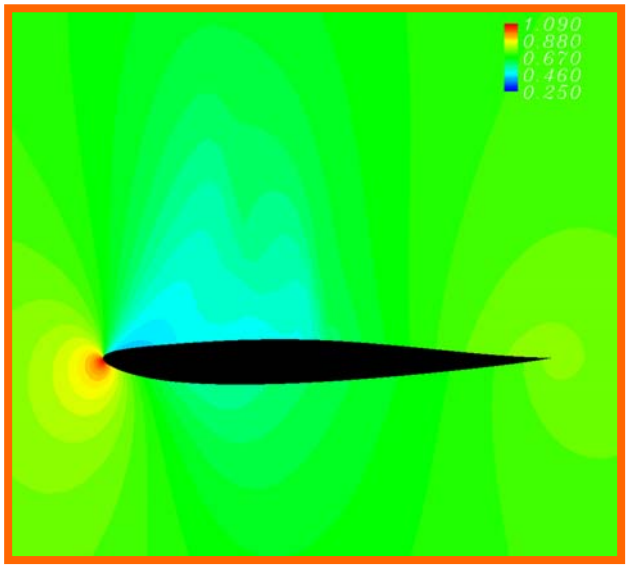
揚力係数最大の解（圧力分布）
Lift maximum design

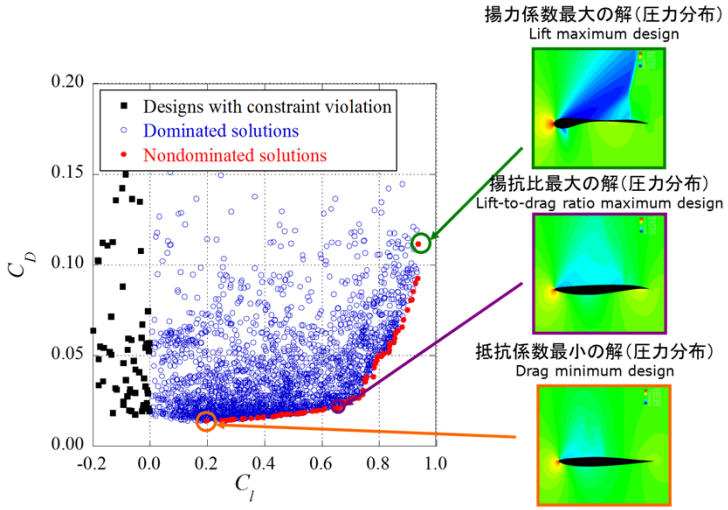
揚抗比最大の解（圧力分布）
Lift-to-drag ratio maximum design

抵抗係数最小の解（圧力分布）
Drag minimum design



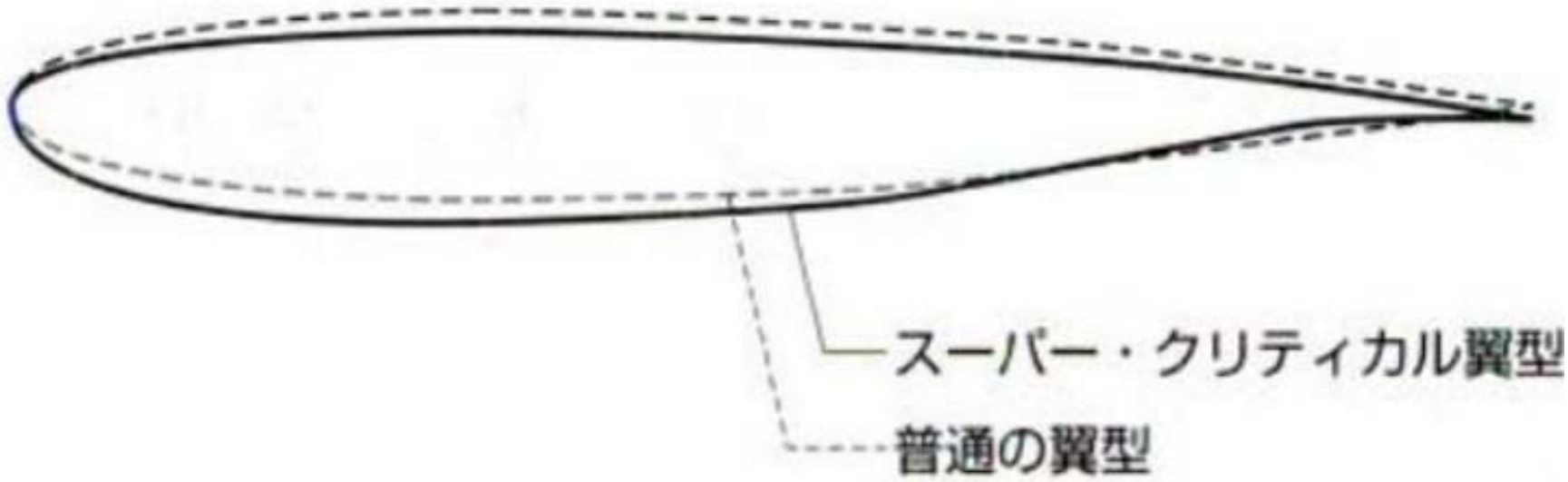



# スーパークリティカル翼 supercritical airfoil

- 前縁から後縁に向かって翼上面側はなるべく平板に近い形
  →衝撃波の発生する位置が翼面の後縁部にくるようにして
  急な剥離を遅らせる
  Upper surface is flat to maintain shock near trailing edge
- 翼下面は大きな膨らみを持っている　→翼厚確保
  Lower surface is designed to maintain enough thickness
- 後縁部にキャンバーをもたせる　→揚力確保
  Strong camber near the trailing edge to obtain high lift



# 設計最適化手法

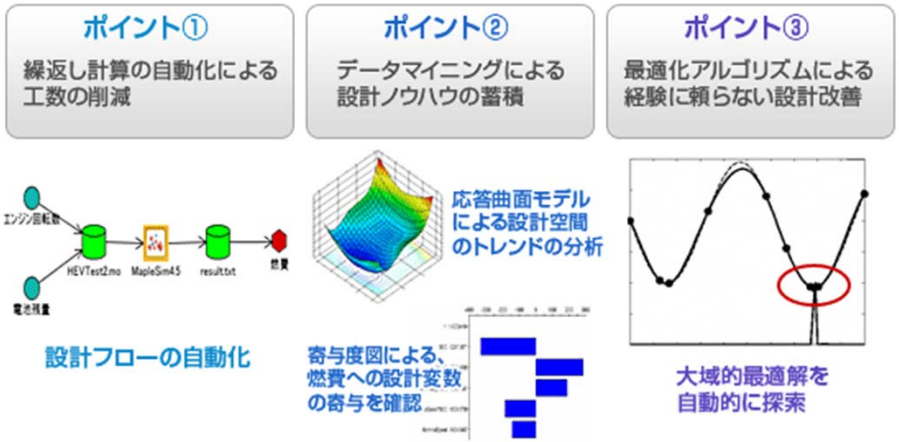
Design optimization methods

# 市販の設計最適化ソフトウェア Commercial software

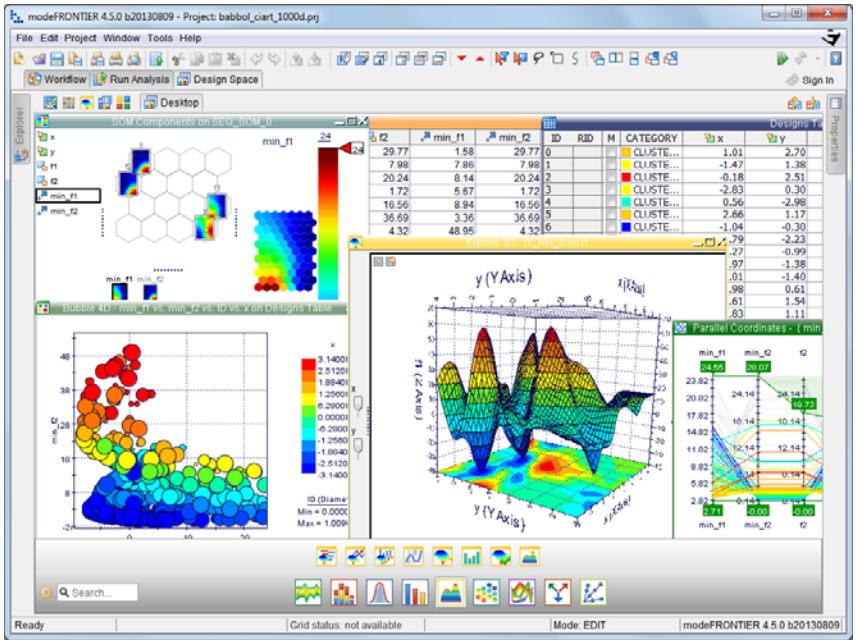
## isight



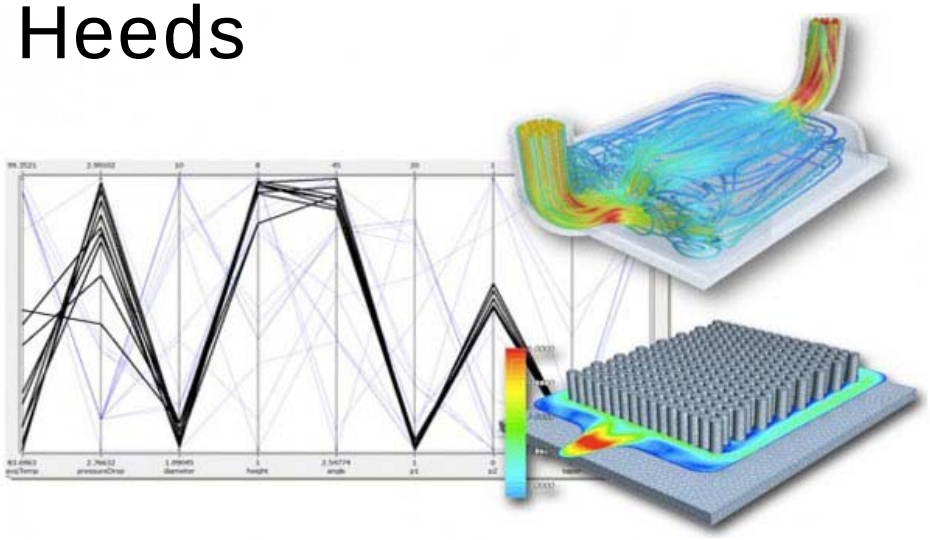

## Optimus



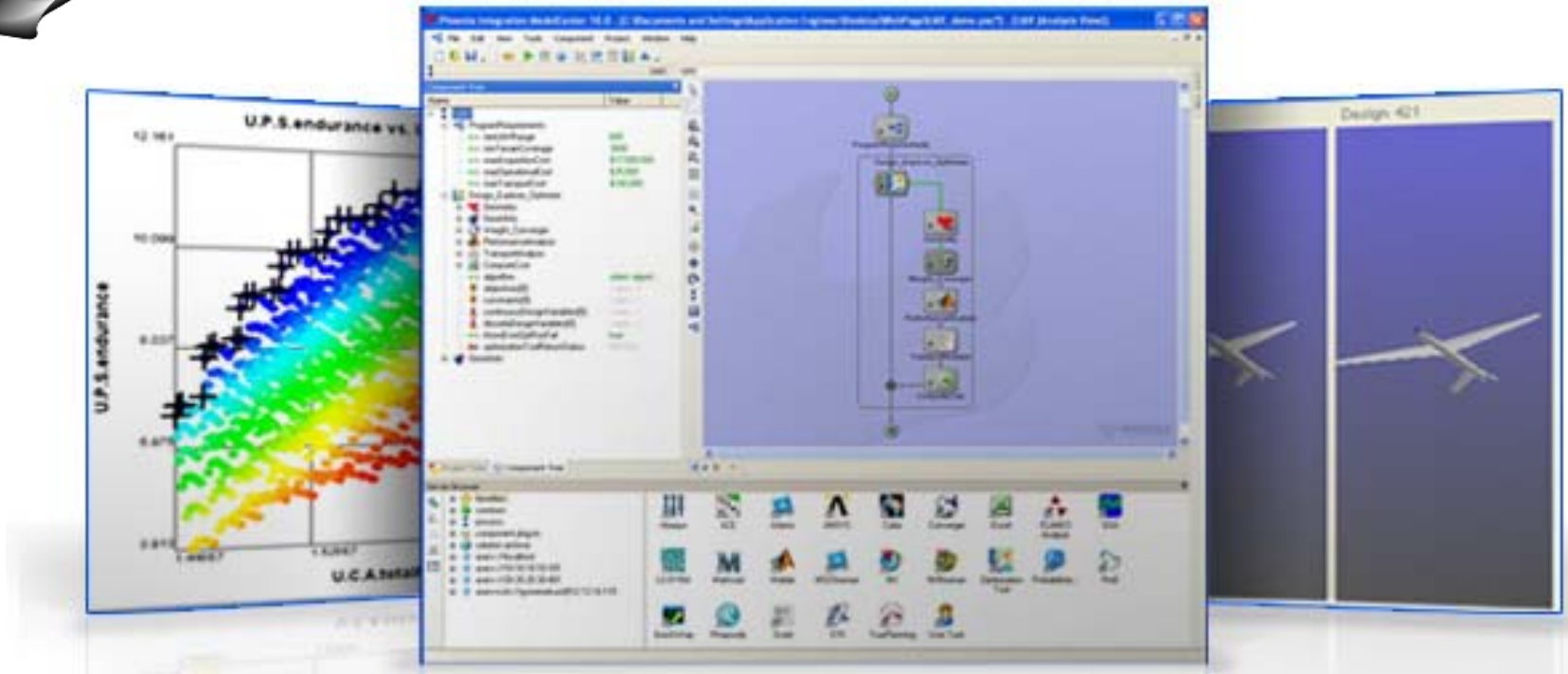

## modeFRONTIER




## Heeds

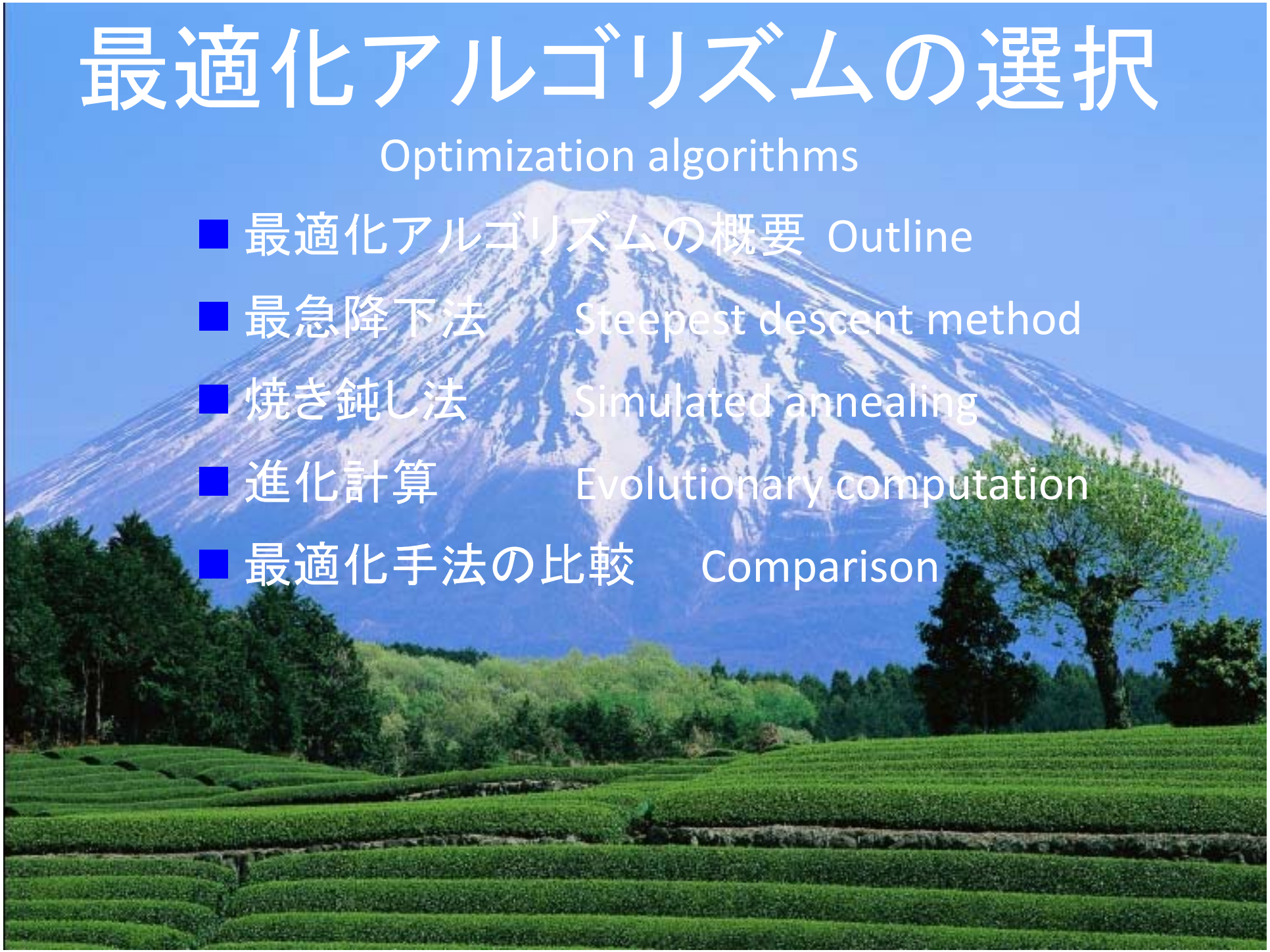


# オープンソースの設計最適化ソフトウェア Dakota



# DAKOTAに入っている最適化アルゴリズム

- 実行可能方向法(Feasible Directions Method)
- Fletcher-Reeves共役勾配法
- BFGS(Broyden-Fletcher-Goldfarb-Shanno)法
- 逐次線形計画法(SLP)
- 逐次二次計画法(SQP)
- Polak-Ribiere共役勾配法
- ニュートン法、準ニュートン法、有限差分ニュートン法
- パターンサーチ
- Simplex法
- グリーディ算法に基づく発見的探索法
- 遺伝的アルゴリズム(JEGA)
- ノースカロライナ州立大学DIRECT
- PDS法(Plan-Do-See)
- EGO(Efficient Global Optimization)



# 最適化アルゴリズムの選択

## Optimization algorithms

- 最適化アルゴリズムの概要 Outline
- 最急降下法 Steepest descent method
- 焼き鈍し法 Simulated annealing
- 進化計算 Evolutionary computation
- 最適化手法の比較 Comparison

# 設計変数での分類 category of problems

- 実数変数最適化問題

  Real variable optimization problem
- 離散変数最適化問題

  Discrete variable optimization problem
- 分類変数最適化問題

  Categorical variable optimization problem
- 混合変数最適化問題

  Mixed variable optimization problem

# 実数変数最適化問題 Real variable optimization problem

- 線形計画問題 Linear programming problem
  - 定義：目的関数および制約条件関数が線型関数である問題

    objective and constraint functions are linear
  - 解法：シンプレックス法など simplex method, etc

- 2次計画問題 Quadratic programming problem
  - 定義：目的関数が2次式で定義され、制約条件関数が線型関数によって定義されている問題

    Objective function is quadratic & constraint function is linear
  - 解法：双対法など Dual method, etc

- 非線形最適化問題(非線形計画問題） Non-linear optimization problem
  - 目的関数and/or制約条件関数が非線形である問題

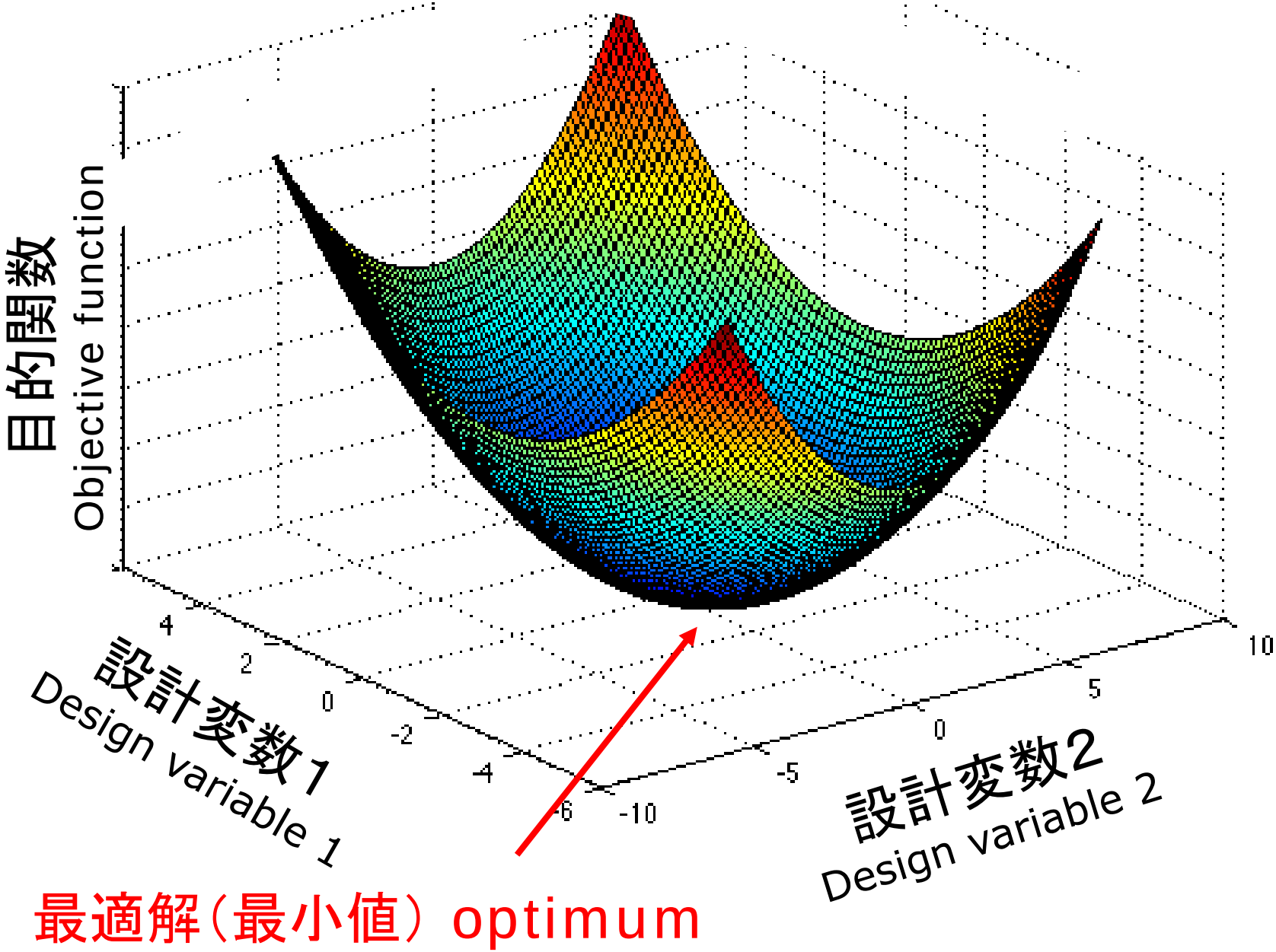
    Objective function and/or constraint function are non-linear
  - 解法：勾配法，進化計算など gradient method, evolutionary computation, etc

# 非線形実数変数最適化問題

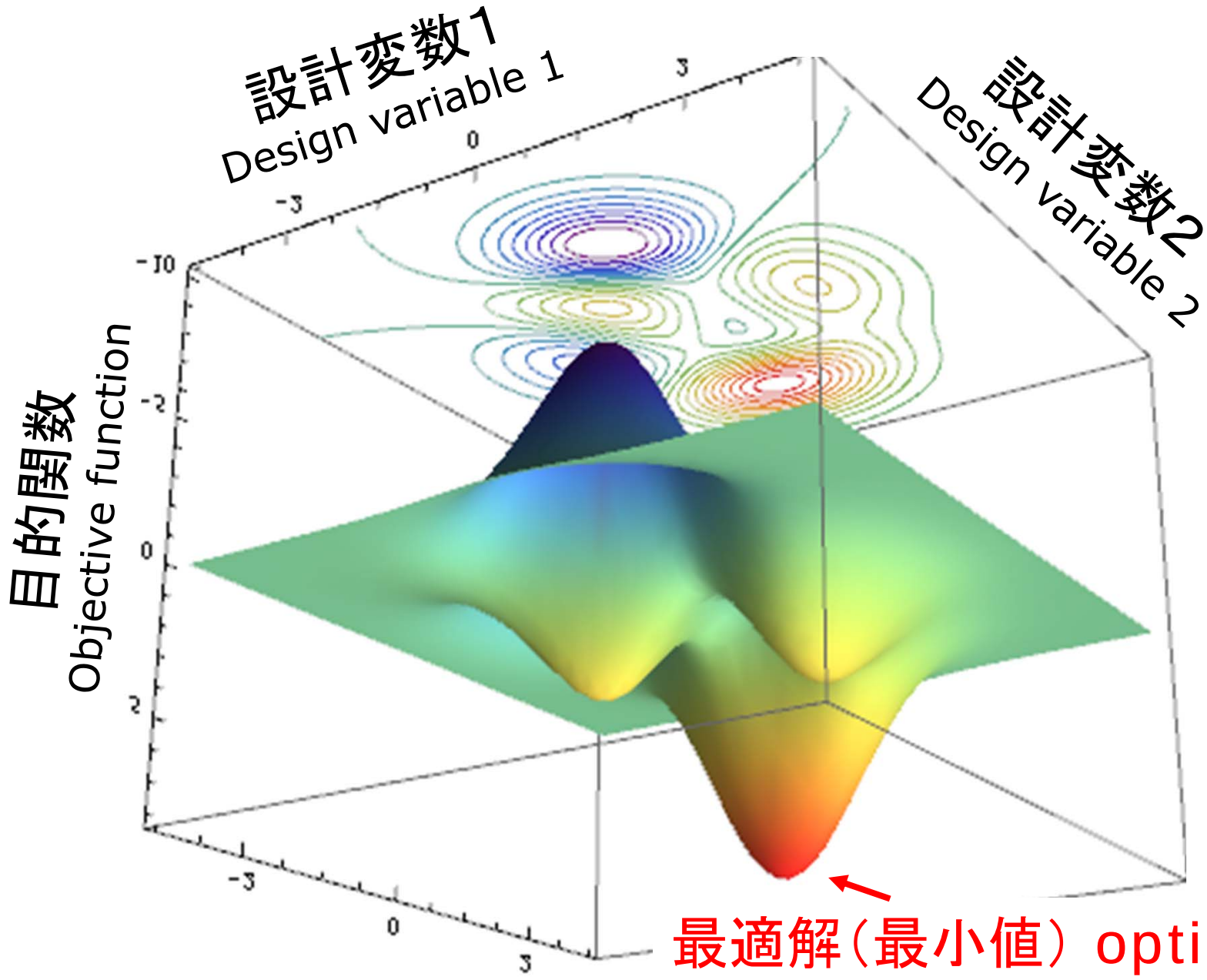
## どんな問題を解きたいのか？

What kind of problem we want to solve?

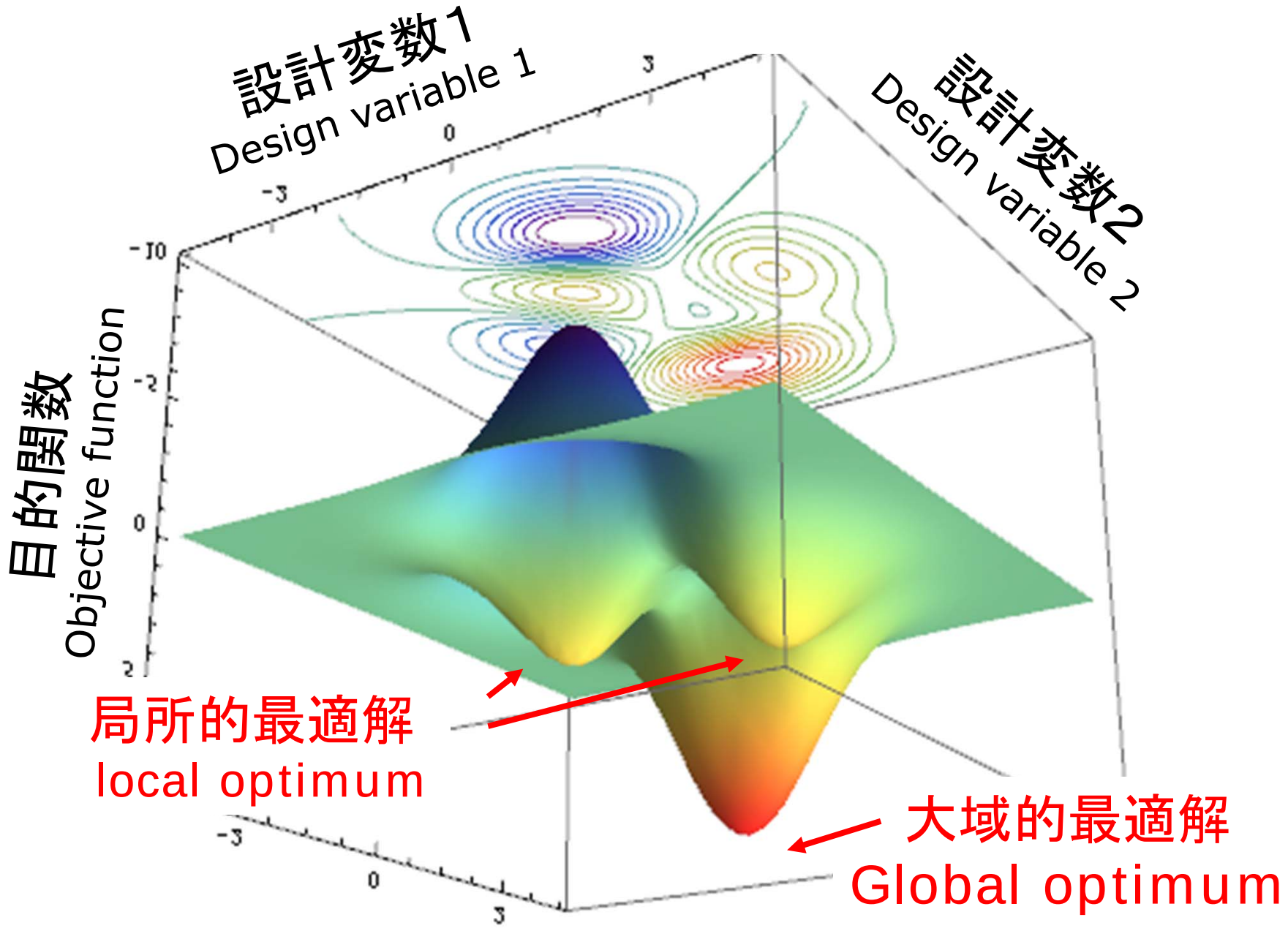
# 単峰的関数 unimodal function

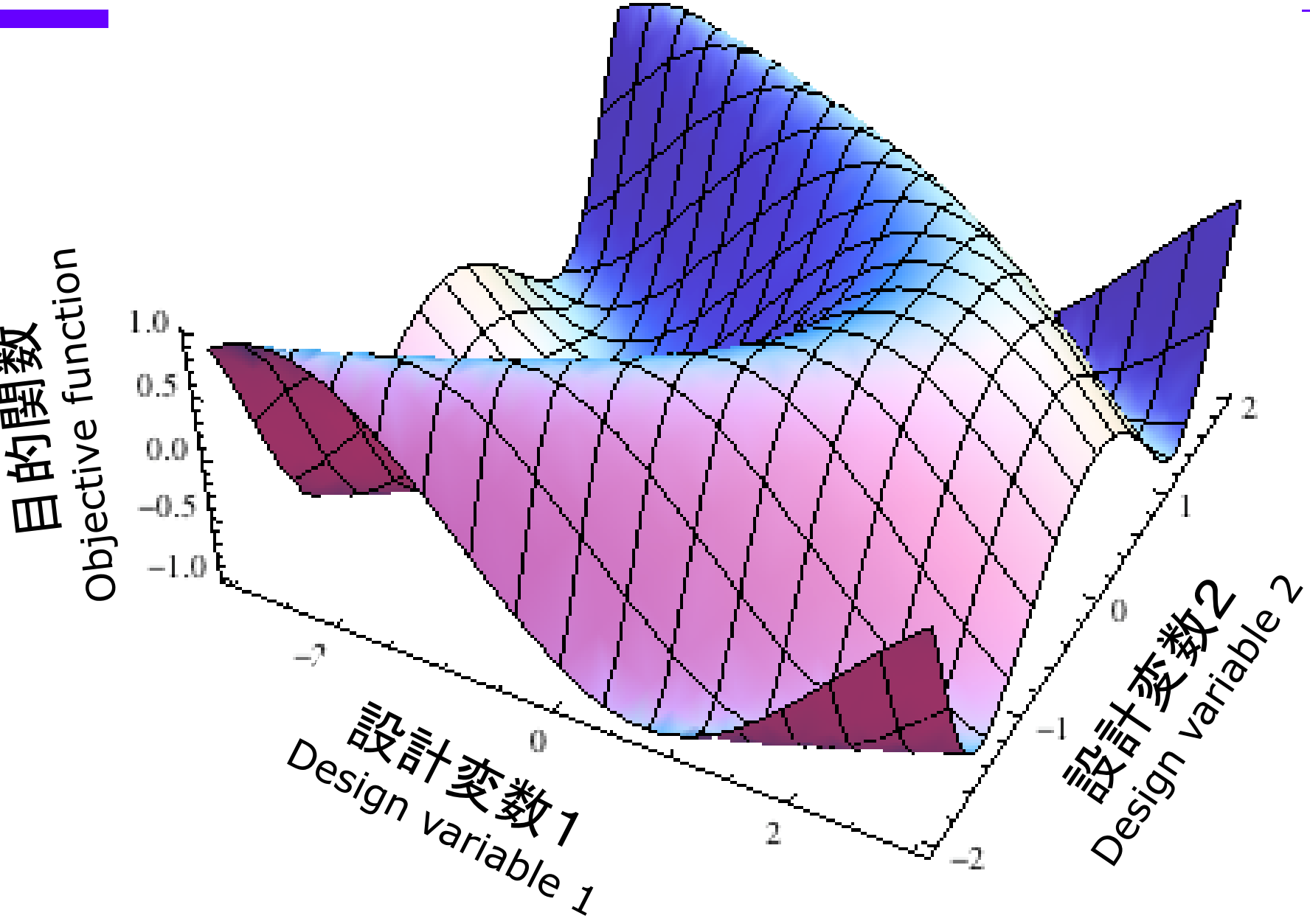


# 多峰的関数 multimodal function



# 多峰的関数 multimodal function




目的関数
Objective function
1.0
0.5
0.0
−0.5
−1.0
−2
0
2
設計変数1
Design variable 1
−2
−1
0
1
2
設計変数2
Design variable 2

# 非線形最適化問題の解法 Solving non-linear problems

- 決定論的最適化手法 deterministic optimization methods
  - 最適化に乱数を用いない algorithm does not base on random numbers
  - 初期解を決めればいつでも同じ解が出る same solution for an initial design
  - 代表的な手法 typical approaches
    - 勾配法 (gradient-based methods)
      - 最急降下法 (steepest descending method)
      - 逐次2次計画法 (sequential quadratic programming)
      - . . .
- 確率論的最適化手法 stochastic optimization methods
  - 最適化に乱数を用いる algorithm bases on random numbers
  - 初期解を決めてもいつも同じ解がでるとは限らない not same solution for an initial design
  - 代表的な手法 typical approaches
    - 焼き鈍し法 (simulated annealing)
    - 進化アルゴリズム (evolutionary algorithm)

# 最急降下法

steepest descending method

# 最急降下法　steepest descending method

- 1847年にCauchyによって開発された

  Developed in 1847 by Cauchy

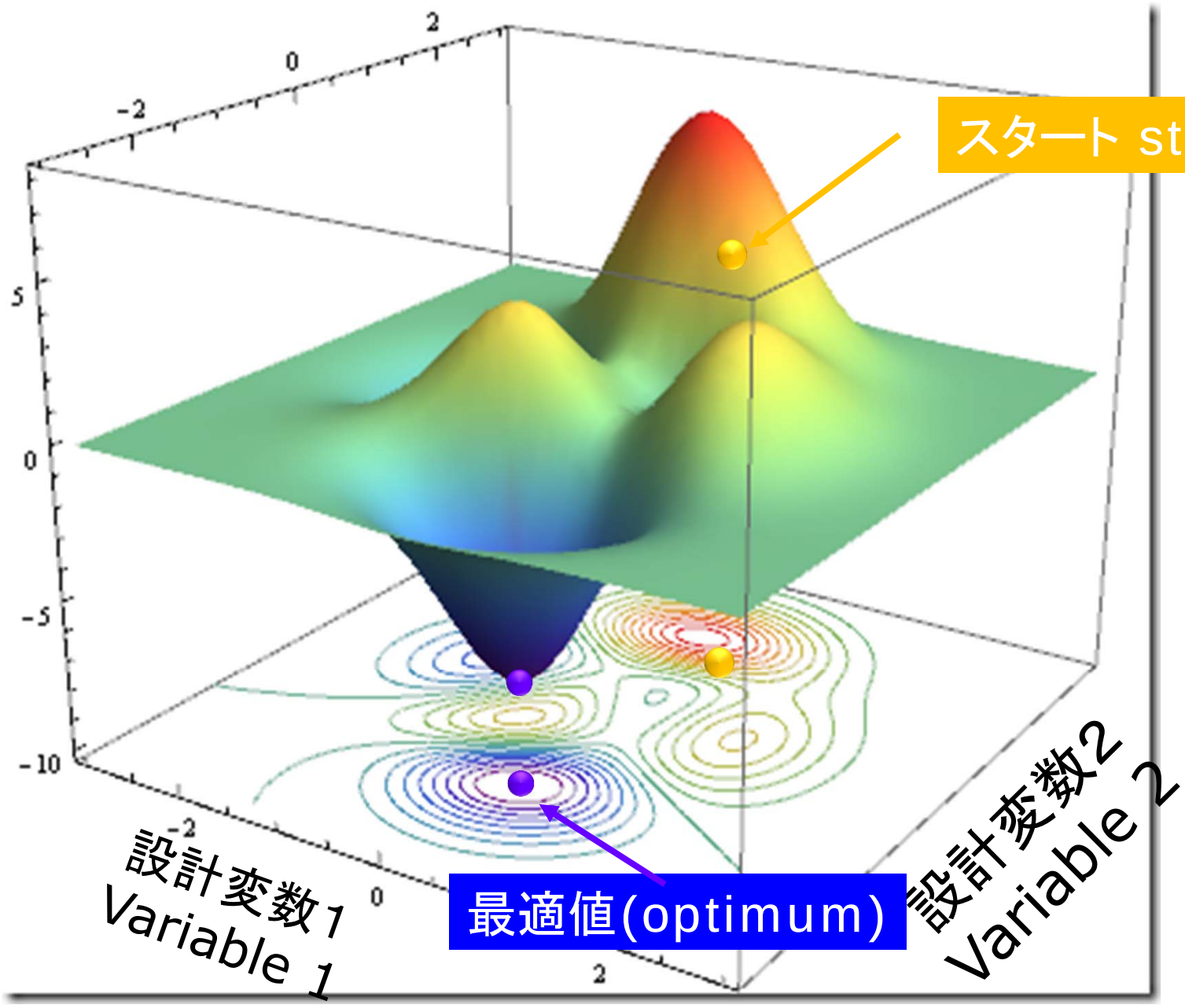
- 設計変数が複数あり，そのすべてが連続変数で，目的関数が1つ，制約条件のない場合に用いられる手法.

  For problems with multiple continuous design variables and one design objective without any constraints

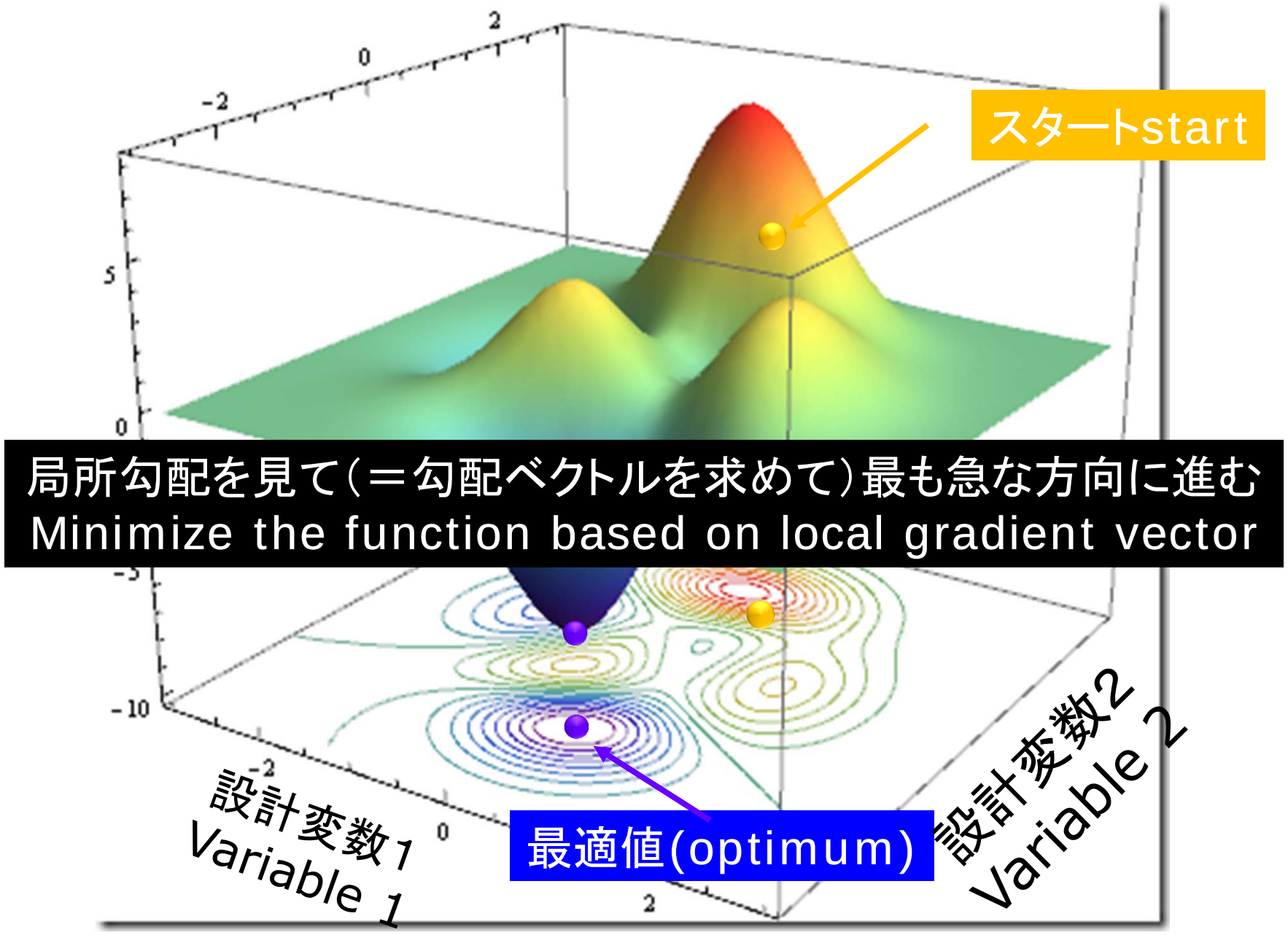
- 目的関数の傾斜方向に沿って目的関数を下げていく

  Minimize the objective function based on gradient of the function.

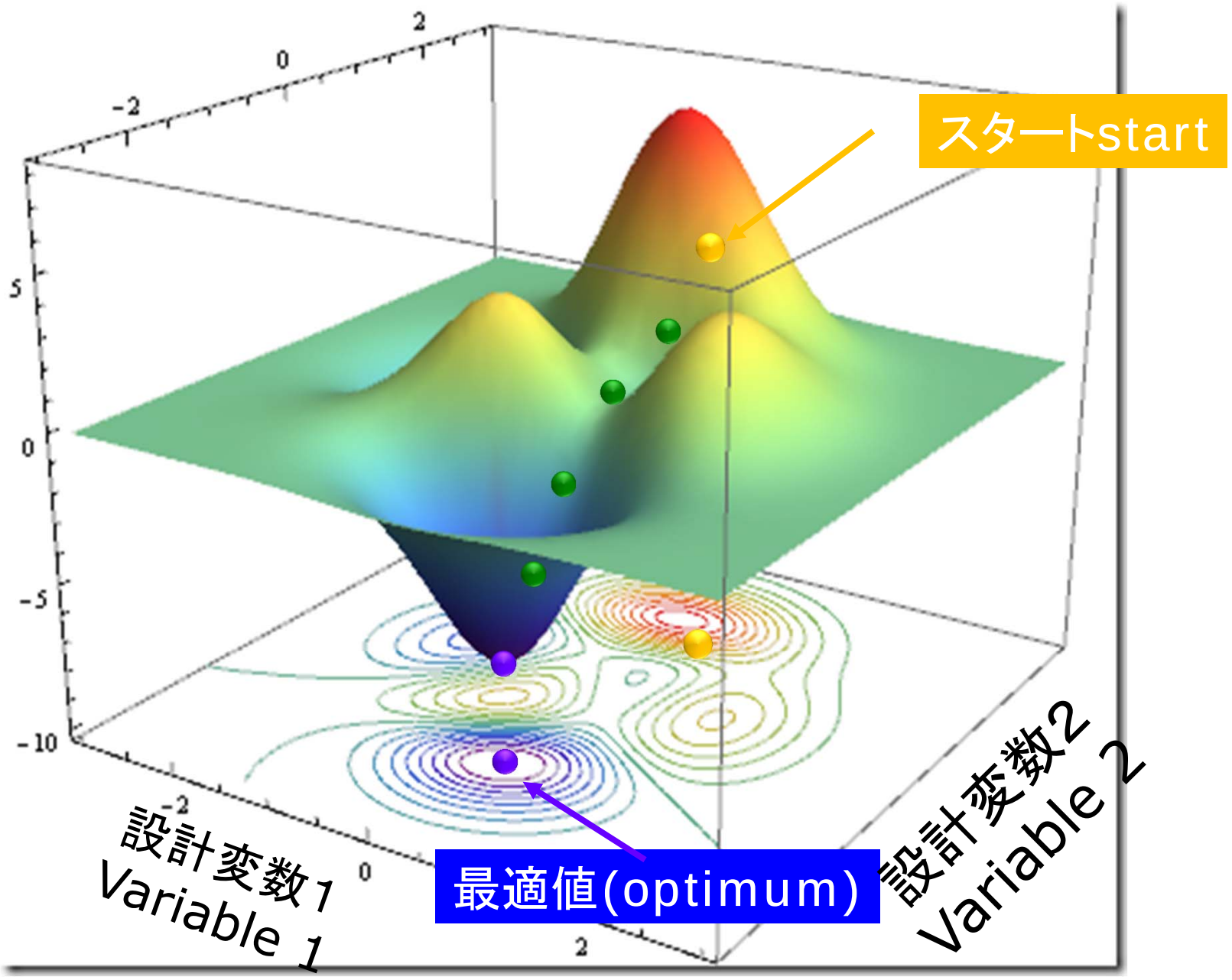
# 最急降下法 steepest descending method



# 最急降下法 steepest descending method



# 最急降下法 steepest descending method



# 勾配ベクトルの求め方 how to obtain gradient vector

関数 $f$ の勾配ベクトル gradient vector of function $f$

$$\nabla f(\vec{x}) = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_N} \right)$$

１変数関数の場合 in case of one variable

$$f(x) = x^2 \qquad \nabla f(x) = \frac{\partial f}{\partial x} \qquad \nabla f(x) = 2x$$

２変数関数の場合 in case of two variables

$$f(x_1, x_2) = \sin x_1 + \cos x_2$$

$$\nabla f(\vec{x}) = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) \qquad \nabla f(\vec{x}) = \left( \cos x_1, \quad -\sin x_2 \right)$$

# 勾配法の基本的な考え方 basic idea

(1) 勾配ベクトルを計算する

compute gradient vector

$$\vec{S}_i = \nabla f(\vec{x}^i) = \left( \frac{\partial f}{\partial x_1^i}, \frac{\partial f}{\partial x_2^i}, \ldots, \frac{\partial f}{\partial x_N^i} \right)$$

(2) 新しい解を作成する

generate new solution

$$\vec{x}^{i+1} = \vec{x}^i + \tau_i \vec{S}_i$$ （最大化の場合） (maximization)

$$\vec{x}^{i+1} = \vec{x}^i - \tau_i \vec{S}_i$$ （最小化の場合） (minimization)

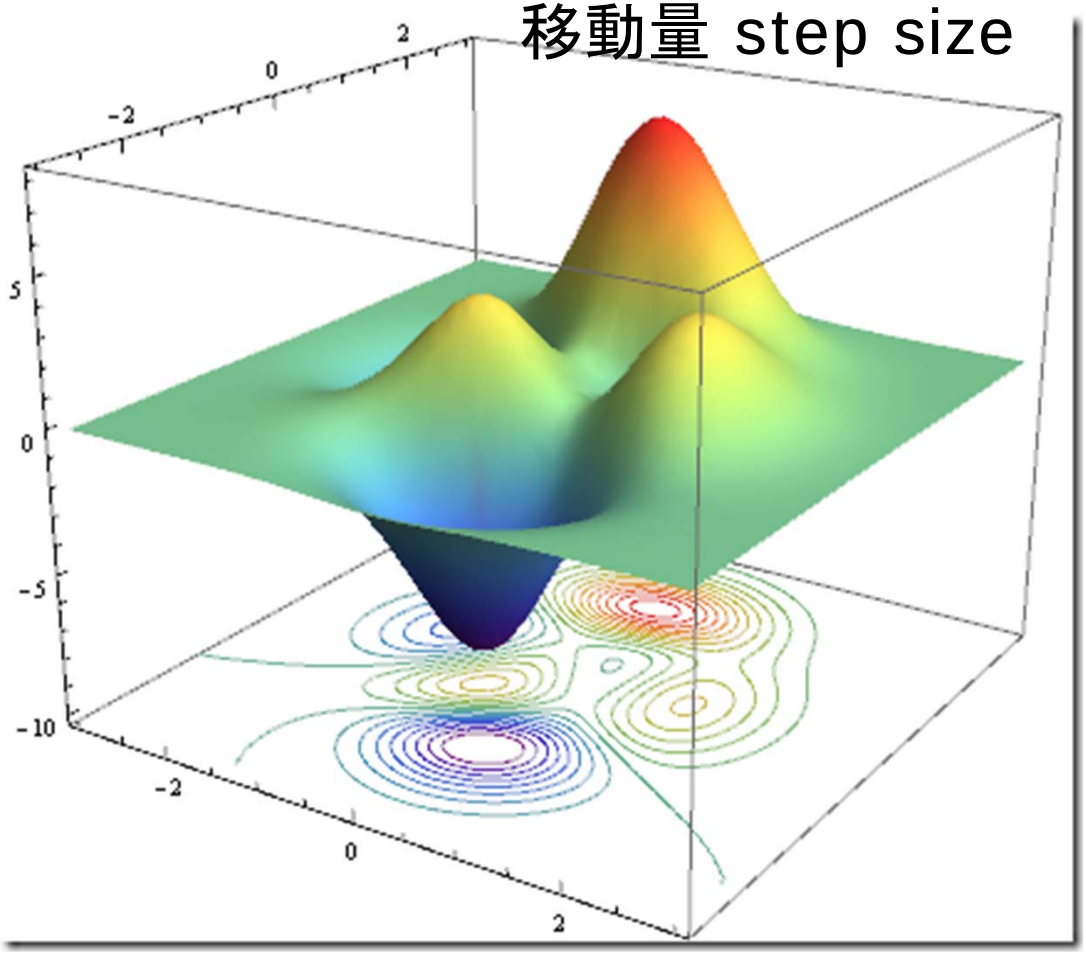
ステップ幅 step width

# 移動量（ステップ幅）の決定 how to define step size

$$x^{(i+1)} = x^{(i)} + \alpha\, d^{(i)}$$

勾配 gradient

移動量 step size



# 移動量(ステップ幅)の決定how to define step size

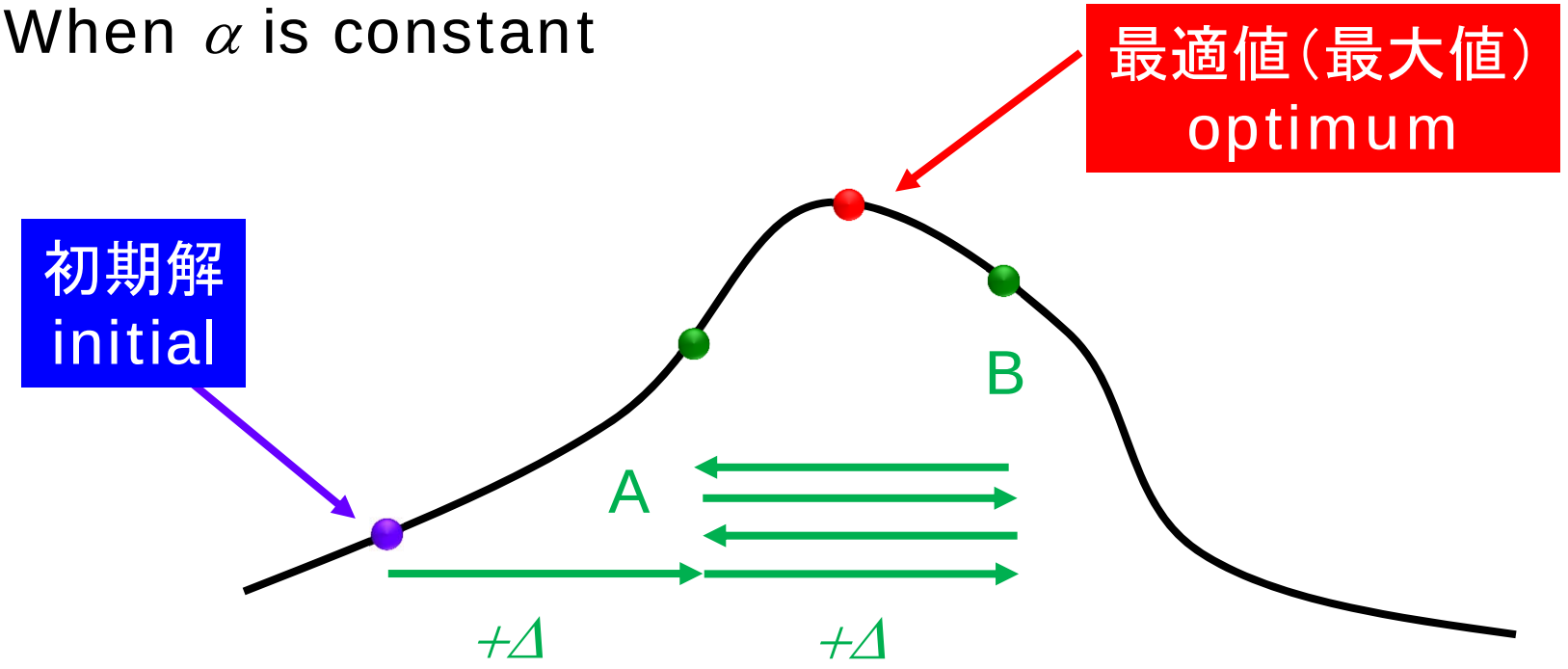
$$x^{(i+1)} = x^{(i)} + \alpha\, d^{(i)}$$

勾配 gradient

移動量 step size

α を一定とすると. . .

When $\alpha$ is constant



# 移動量（ステップ幅）の決定 how to define step size

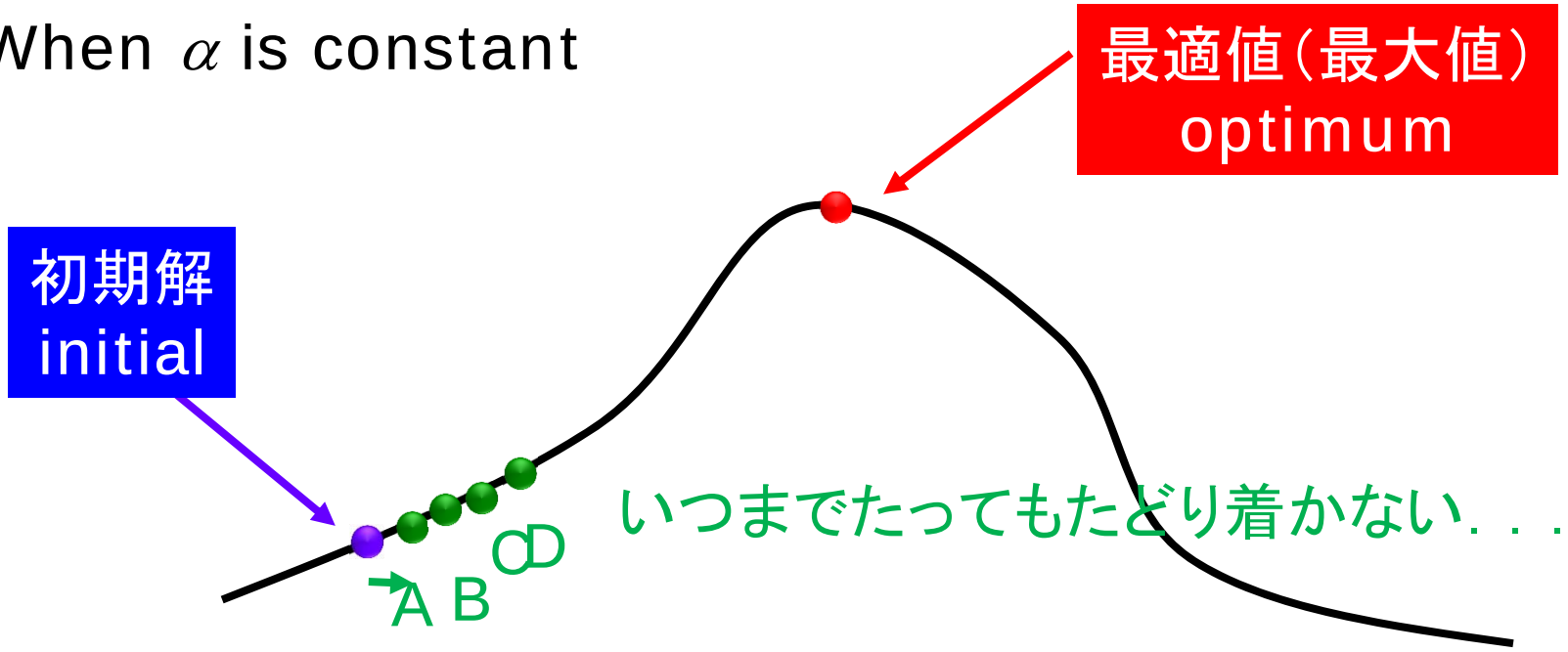
$$x^{(i+1)} = x^{(i)} + \alpha\, d^{(i)}$$

勾配 gradient

移動量 step size

$\alpha$ を一定とすると. . .

When $\alpha$ is constant



# 移動量（ステップ幅）の決定 how to define step size

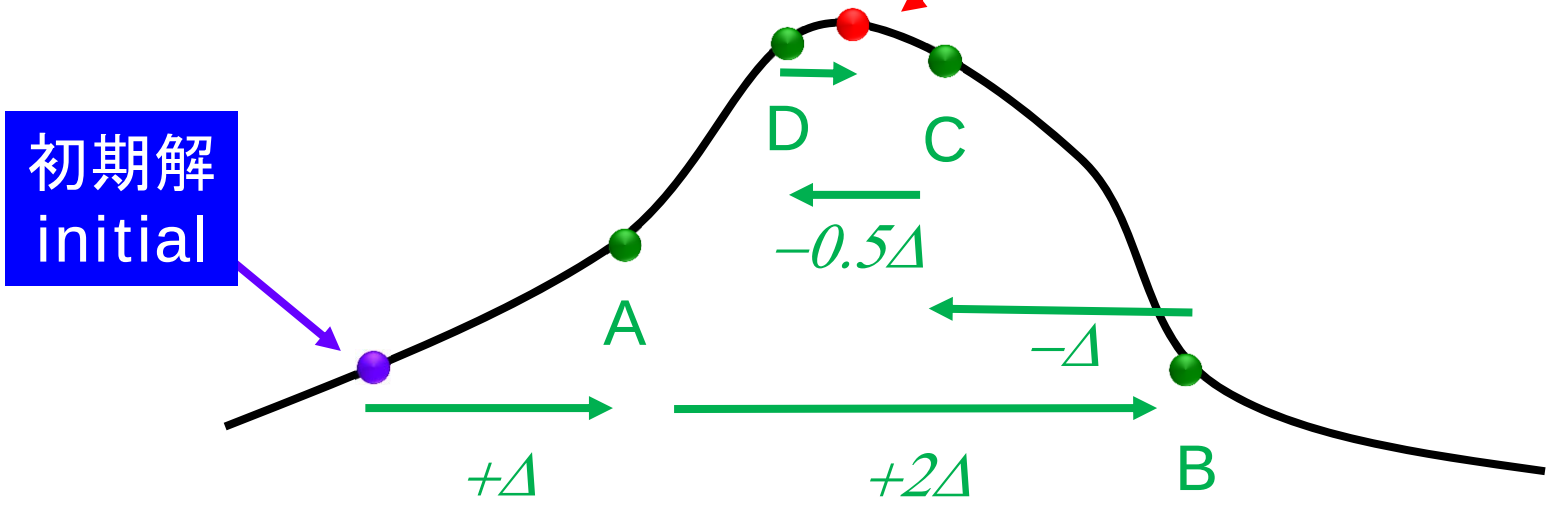
$$x^{(i+1)} = x^{(i)} + \alpha\, d^{(i)}$$

勾配 gradient

移動量 step size

α を適切に定めると．．．

When $\alpha$ is properly changed




適切な移動量をどうやって決めるのか？→一次元探索

How to define step size? -> one-dimensional search

# 移動量（ステップ幅）の決定 how to define step size

$$x^{(i+1)} = x^{(i)} + \alpha\, d^{(i)}$$

勾配 gradient

移動量 step size

1つの方法は勾配に比例させる
Proportional to the gradient

$$\alpha = K\left(\frac{\partial f}{\partial x_j}\right)_{x=x_i}$$

急なところでは移動量を大きく
Large step size when steep
平らなところでは移動量を小さく
Small step size when flat
ただし適切なKを定義することが難しい
Constant K should be defined

# 準ニュートン法(Newton-Raphson method)

1変数の関数の場合について説明する
We think one variable problem
移動した先の傾きがゼロ(つまり極値になる)ためには以下の式が成り立つ必要がある
The following should be satisfied to be local optima

$$\left(\frac{df}{dx}\right)_{x=x_0+\Delta x} \approx \left(\frac{df}{dx}\right)_{x=x_0} + \left(\frac{d^2 f}{dx^2}\right)_{x=x_0} \Delta x = 0$$

この式を満たす移動量は step size satisfying this equation is

$$\Delta x = -\left(\frac{df}{dx}\right)_{x=x_0} \Big/ \left(\frac{d^2 f}{dx^2}\right)_{x=x_0}$$

※ただしこれだけではだめ. なぜか？
however, this is not enough. Why?

# 準ニュートン法(Newton-Raphson method）

$$\left(\frac{df}{dx}\right)_{x=x_0+\Delta x} \approx \left(\frac{df}{dx}\right)_{x=x_0} + \left(\frac{d^2 f}{dx^2}\right)_{x=x_0} \Delta x = 0$$

テイラー展開，つまり，微小な変化量を仮定
Tayler expansion, i.e., small difference is assumed.

$$\Delta x = \min\left(\Delta x_{\max}, -\left(\frac{df}{dx}\right)_{x=x_0} \Big/ \left(\frac{d^2 f}{dx^2}\right)_{x=x_0}\right)$$

ここで，$\Delta x_{\max}$ は小さな値
Where $\Delta x_{\max}$ should be a small value

# 準ニュートン法(Newton-Raphson method）

２変数の関数の場合は

In case of two variables

移動した先の傾きがゼロ（つまり極値になる）ためには以下の連立方程式が成り立つ必要がある

The following equations should be satisfied

$$\left(\frac{\partial f}{\partial x_1}\right)_{x=x_0} + \left(\frac{\partial}{\partial x_1}\frac{\partial f}{\partial x_1}\Delta x_1 + \frac{\partial}{\partial x_2}\frac{\partial f}{\partial x_1}\Delta x_2\right)_{x=x_0} = 0$$

$$\left(\frac{\partial f}{\partial x_2}\right)_{x=x_0} + \left(\frac{\partial}{\partial x_1}\frac{\partial f}{\partial x_2}\Delta x_1 + \frac{\partial}{\partial x_2}\frac{\partial f}{\partial x_2}\Delta x_2\right)_{x=x_0} = 0$$

# 準ニュートン法(Newton-Raphson method）

n変数の関数の場合は
In case of n variables
移動した先の傾きがゼロ（つまり極値になる）ためには以下の連立方程式が成り立つ必要がある
The following equations should be satisfied

$$\left(\frac{\partial f}{\partial x_i}\right)_{x=x_0} + \sum_{j=1}^{n}\left(\frac{\partial^2 f}{\partial x_i \partial x_j}\right)_{x=x_0} \Delta x_j = 0$$

$$i=1,2,\ldots,n$$

最大値付近での探査速度が速い
設計変数が多い場合は連立方程式を解く労力が大きくなる

# 計算例 example

次の関数の最小値を最急降下法を使って求めよ.

$$f(x)=x^4-16x^3+88x^2-190x+144$$

勾配ベクトルは gradient vector is

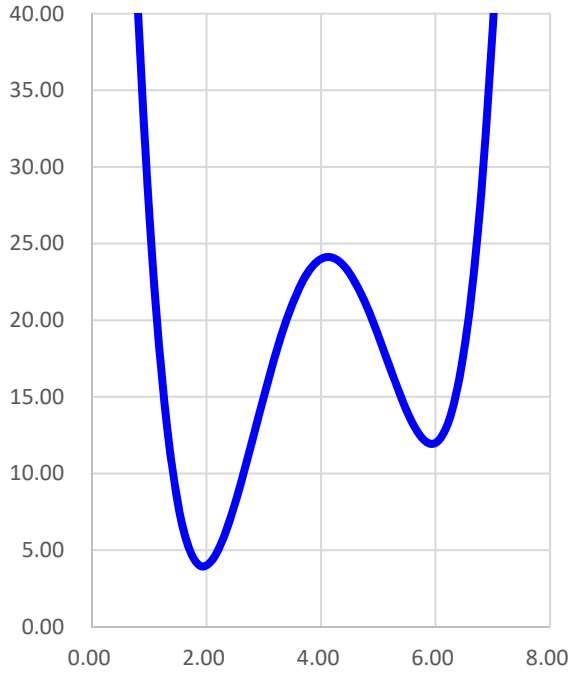
$$S=\nabla f(\vec{x})=\left(\frac{\partial f}{\partial x_1},\frac{\partial f}{\partial x_2},\ldots,\frac{\partial f}{\partial x_N}\right)$$

ステップ幅は step size is

$$\Delta x=\left|-\left(\frac{df}{dx}\right)_{x=x_0}\Big/\left(\frac{d^2f}{dx^2}\right)_{x=x_0}\right|$$

解の更新は update solution

$$x^{i+1}=x^i-S\Delta x \quad \text{（最小化なので）}$$



# 計算例 example

次の関数の最小値を最急降下法を使って求めよ.

$$f(x)=x^4-16x^3+88x^2-190x+144$$

勾配ベクトルは gradient vector is

$$\nabla f(x)=\frac{\partial f}{\partial x}=4x^3-48x^2+176x-190$$

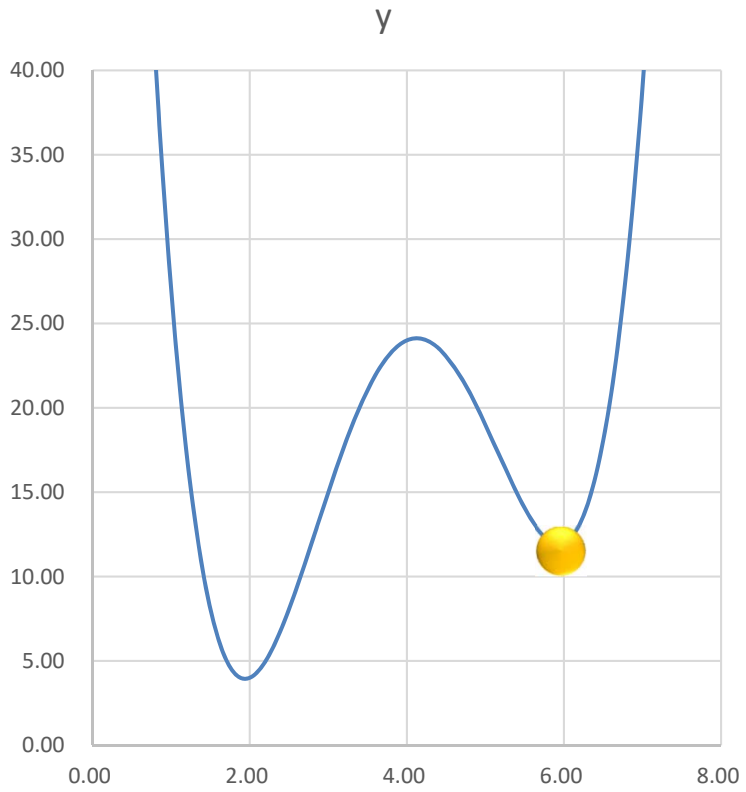
ステップ幅は step size is

$$\Delta x=\left|-\left(\frac{df}{dx}\right)_{x=x_0}\Big/\left(\frac{d^2f}{dx^2}\right)_{x=x_0}\right|=\left|\frac{4x^3-48x^2+176x-190}{12x^2-96x+176}\right|$$

解の更新は update solution

$$x^{i+1}=x^i-S\Delta x \quad \text{（最小化なので）}$$

# Excel file GM(1)




| initial solution | optimized solution |
|---|---|
| X=1 | diverged |
| X=2 | diverged |
| X=3 | diverged |
| X=4 | diverged |
| X=5 | diverged |
| X=6 | 5.93 |
| X=7 | diverged |
| X=8 | diverged |
| X=9 | diverged |
| X=10 | diverged |

ステップ幅が大きすぎると発散する（傾きが大きい場合）

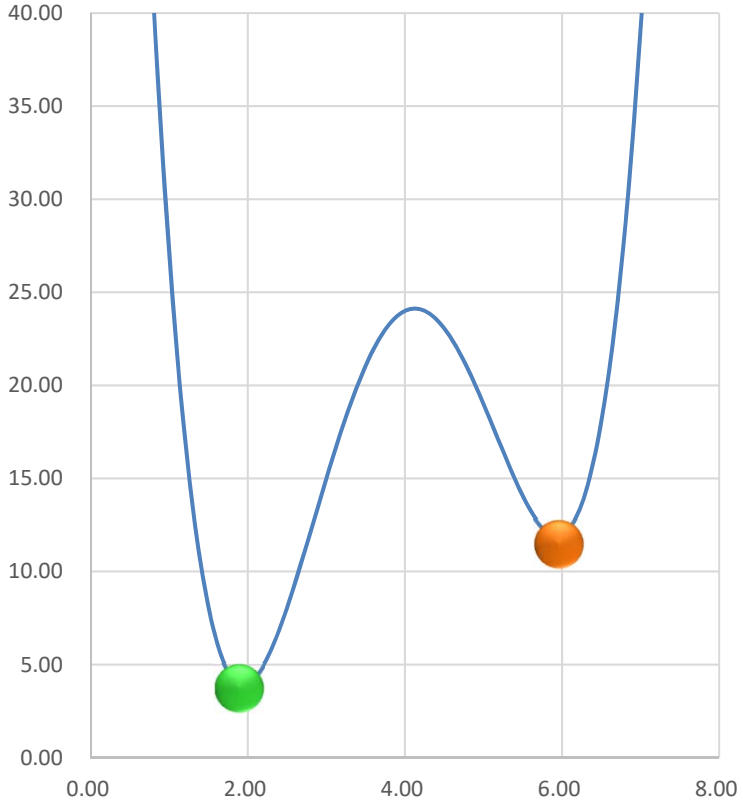
When step size is too large, the computation diverges

$$\Delta x = \left| -\left(\frac{df}{dx}\right)_{x=x_0} \Big/ \left(\frac{d^2 f}{dx^2}\right)_{x=x_0} \right| = \left| \frac{4x^3 - 48x^2 + 176x - 190}{12x^2 - 96x + 176} \right|$$

# Excelファイル　GM(2)

ステップ幅を制限 limit the step size

$$\Delta x = \begin{cases} |\Delta x| & if \quad |\Delta x| < 0.01 \\ 0.01 & if \quad |\Delta x| \geq 0.01 \end{cases}$$




| initial solution | optimized solution |
|---|---|
| x=1 | 1.94 |
| x=2 | 1.94 |
| X=3 | 1.94 |
| X=4 | 1.94 |
| X=5 | 5.93 |
| X=6 | 5.94 |
| X=7 | 5.94 |
| X=8 | 5.94 |
| X=9 | 5.93 |
| X=10 | 1.94 |

# アルゴリズム steepest descending method algorithm

(1) 初期解の設計変数値を定める $\vec{x}^0$

give an initial design

(2) 目的関数値とその勾配を計算する $\vec{S}_i = -\nabla f(\vec{x}^i) = \left( \frac{\partial f}{\partial x_1^i}, \frac{\partial f}{\partial x_2^i}, \ldots, \frac{\partial f}{\partial x_N^i} \right)$

compute objective function and its gradient

(3) ステップ幅を求める

compute step size

(4) 新しい解を作成する $\vec{x}^{i+1} = \vec{x}^i + \tau_i \vec{S}_i$

$\tau_i$ ステップ幅

generate new solution

(5) 収束条件が満たされれば最適解が発見されたとして終了.
そうでなければ, ステップ2にもどる

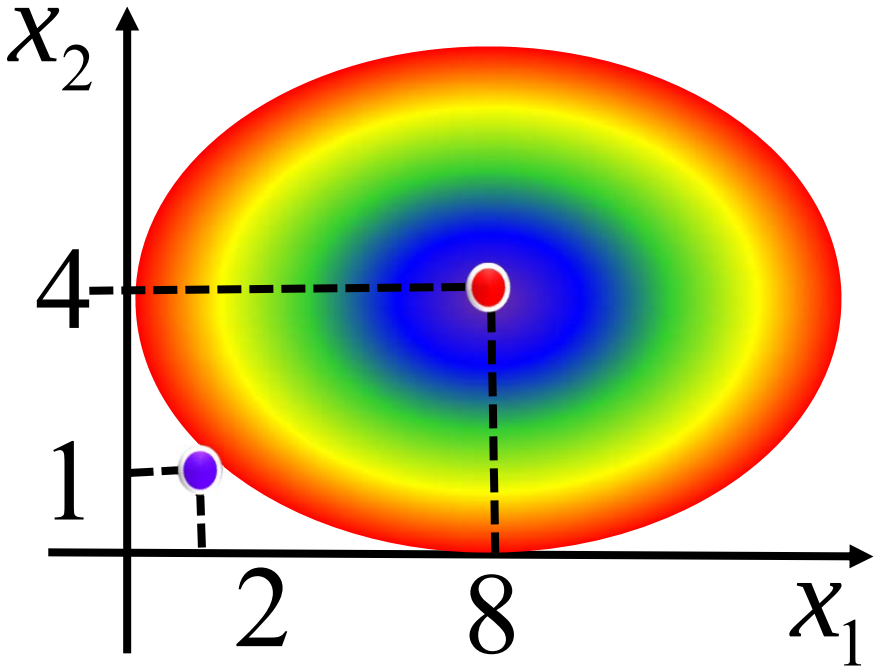
stop if the convergence conditions are satisfied. Otherwise go to step (2)

$$\left| \frac{f\left(\vec{x}^{i+1}\right) - f\left(\vec{x}^i\right)}{f\left(\vec{x}^i\right)} \right| \le \varepsilon_1 \qquad \left| \frac{\partial f}{\partial x_j^i} \right| \le \varepsilon_2 \qquad \left| \vec{x}^{i+1} - \vec{x}^i \right| \le \varepsilon_3$$

# 演習 practice

次の関数の最適値（最小値）を最急降下法で求めてみよう
Obtain maximum using steepest descending method

$$f(x_1, x_2) = x_1^2 - 16x_1 + 2x_2^2 - 16x_2$$




i番目の探索点の次の探索点の
j番目の成分は

$$x_{j,i+1} = x_{j,i} + K\left(\frac{\partial f}{\partial x_j}\right)_{x=x_i}$$

K=0.2とする

*初期値を$(x_1,x_2)=(2,1)$とする*
Initial solution is (2,1)

# 演習 practice

$$f(x_1, x_2) = x_1^2 - 16x_1 + 2x_2^2 - 16x_2$$

$$\frac{\partial f}{\partial x_1} = 2x_1 - 16 \qquad \frac{\partial f}{\partial x_2} = 4x_2 - 16$$

初期値は initial solution is ( $x_{1,0}$, $x_{2,0}$ )=*(2,1)*

$$f_0 = f(2,1) = -42$$

1回目の探索点は first solution is

$$x_{1,1} = x_{1,0} - K(\partial f / \partial x_1) = 2 + 0.2 \times 12 = 4.4$$

$$x_{2,1} = x_{2,0} - K(\partial f / \partial x_2) = 1 + 0.2 \times 12 = 3.4$$

$$f_1 = f(4.4, 3.4) = -82.32$$

# 演習 practice

２回目の探索点は the second solution is

$$x_{1,2} = x_{1,1} - K(\partial f / \partial x_1) = 4.4 - 0.2 \times (-7.2) = 5.84$$

$$x_{2,2} = x_{2,1} - K(\partial f / \partial x_2) = 3.4 - 0.2 \times (-2.4) = 3.88$$

３回目の探索点は the third solution is

$$x_{1,3} = x_{1,2} - K(\partial f / \partial x_1) = 5.84 - 0.2 \times (-4.32) = 6.704$$

$$x_{2,3} = x_{2,2} - K(\partial f / \partial x_2) = 3.88 - 0.2 \times (-0.48) = 3.976$$

同様に進めていくと(8,4)に収束する
Finally converged to (8,4)

Excel file example2

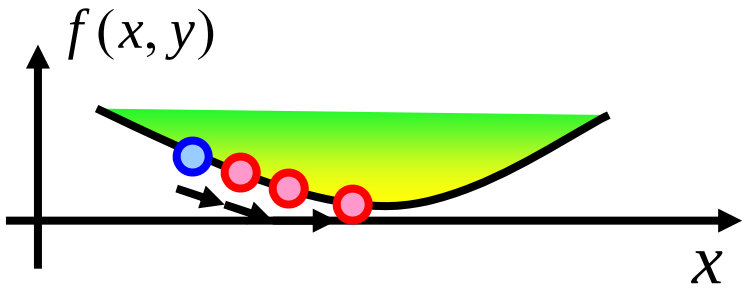
# 最急降下法の利点は？

## Advantage of SDM?

# 最急降下法の利点 advantage of SDM

- 勾配情報を用いるので探査効率で他の手法より優れている
  Efficient

目的関数分布が単峰的である場合
効率的に大域的最適解に至る
Global convergence if uni-modal



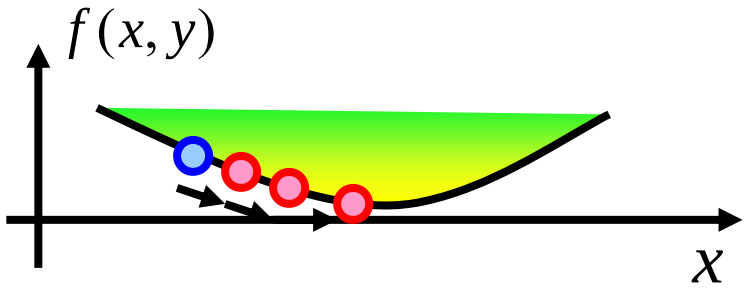
# 最急降下法の欠点は？

## Disadvantage of SDM?

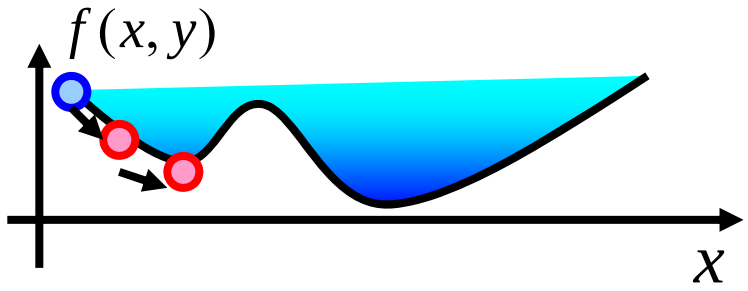
# 欠点 disadvantages of SDM

- 目的関数分布が多峰的である場合，局所解に収束する
  Local convergence if the objective function is multimodal

目的関数分布が単峰的である場合
大域的最適解に至る
Global convergence if uni-modal



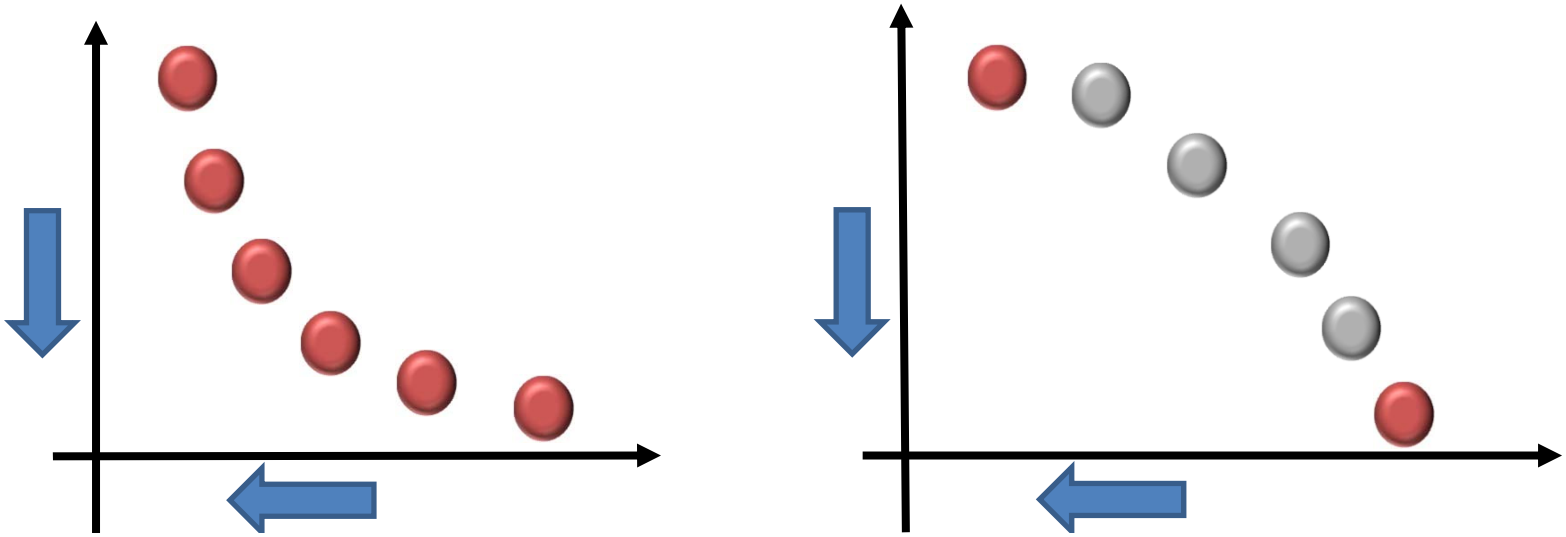

目的関数分布が多峰的である場合
局所解に至る可能性が高い
Local convergence if multimodal



# 欠点 disadvantages of SDM

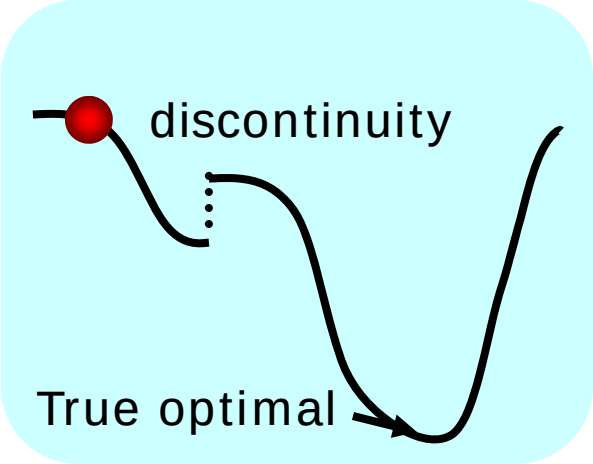
- 目的関数分布が多峰的である場合，局所解に収束する
  Local convergence if the objective function is multimodal
- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない
  Find only of the Pareto-optimal solutions

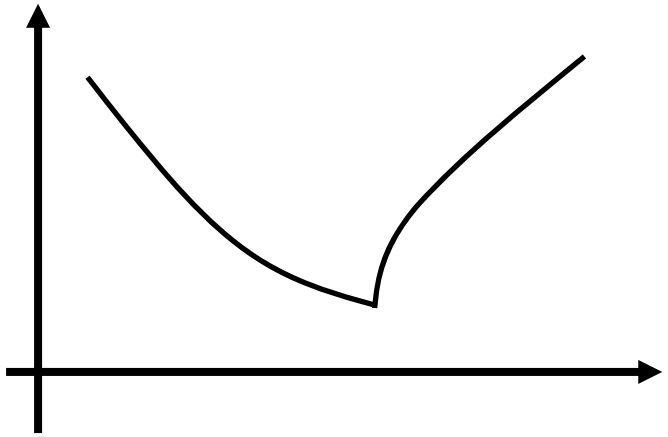
$$f = a \cdot f_1 + (1-a) \cdot f_2$$



# 欠点 disadvantages of SDM

- 目的関数分布が多峰的である場合，局所解に収束する
  Local convergence if the objective function is multimodal
- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない
  Find only of the Pareto-optimal solutions
- 不連続関数最適化問題では計算ができない
  Not applicable to design problems with discontinuous objective function
- 微分不可能関数最適化問題では計算ができない
  Not applicable to design problems with non-differenciable objective function



# 最急降下法の制約付き問題への適用 application to constrained problems

制約付き最適化問題は解くことが出来ない

よって，制約付き問題を制約無し問題に変換する

Constrained problem is transformed to unconstrained problem

最小化 minimize $f(x)$ 制約条件 constraints $g_i(x) \leq 0$



最小化 minimize $f(x) + h(x)$

ここで where $h(x)$

$$\begin{cases} 0 & \text{制約条件を満たす場合 (when constraints are satisfied)} \\ G(x) & \text{制約条件を満たさない場合} \end{cases}$$

ただし，微分不可能になるので滑らかな関数で近似

$h(x)$ should be approximated by smooth function

# 焼き鈍し法

simulated annealing



# 焼き鈍し法とは？ What's simulated annealing?



金属は加熱後，熱平衡状態を保ちながらゆっくり温度を下げると，エネルギー最低の状態を実現

Optimization algorithm that mimics process of metal achieving the state of lowest energy when the temperature is lowered slowly after heating

S. Kirkpatrick、C. D. Gelatt、M. P. Vecchiらが1983年に考案し、1985年に V. Cerny が再発見

S. Kirkpatrick、C. D. Gelatt、M. P. Vecchi proposed in 1983 and re-discovered by V. Cerny in 1985

# 基本的なアイディア basic idea

(1) ある解（A)の近傍に新しい解（B)をランダムに作成し，目的関数値を評価する.

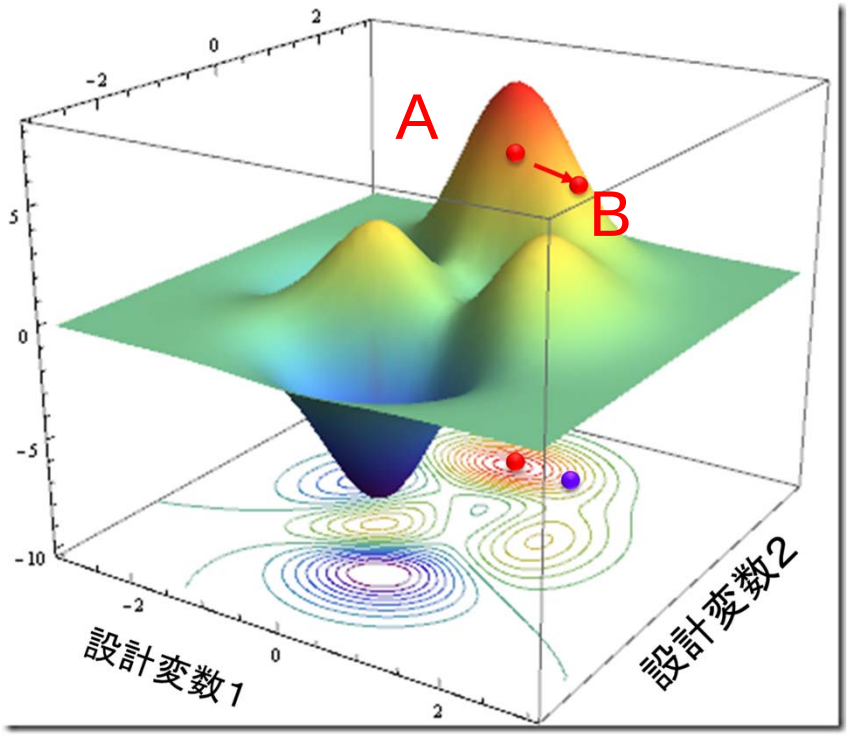
generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)

(2) もとの解と新しい解の目的関数値の差$\Delta E$が負であれば（新しい設計が優れていれば）新しい解を採択

If objective function difference $\Delta E$ is negative (if (B) is better) , take new solution (B)

(3) そうでなければ，確率 $exp(-\Delta E/T(t))$で新しい解を採択（新しい解が優れていなくてもある一定の確率で採択する）

Else take new solution (B) with probability of $exp(-\Delta E/T(t))$



# 基本的なアイディア basic idea

確率 $exp(-\Delta E/T(t))$ で新しい解を採択（新しい解が優れていなくても
ある一定の確率で採択する）

Take new solution (B) with probability $exp(-\Delta E/T(t))$ even if (B) is worse than (A)

$$\Delta E = E_{new} - E_{current}$$

目的関数値の差（常に正）
Objective function difference (positive)

$$T(t) = \alpha^t T_0$$

$\alpha$ 冷却率 cool down rate (0.8～1.0)
$T_0$ 初期温度 initial temperature
$t$ 反復回数 iteration number

$\Delta E / T(t)$ 反復回数が増えるにつれ大きくなる
Increases as number of iterations increases
新しい解の目的関数値が大きいと大きくなる
Increases as objective function of new solution is larger

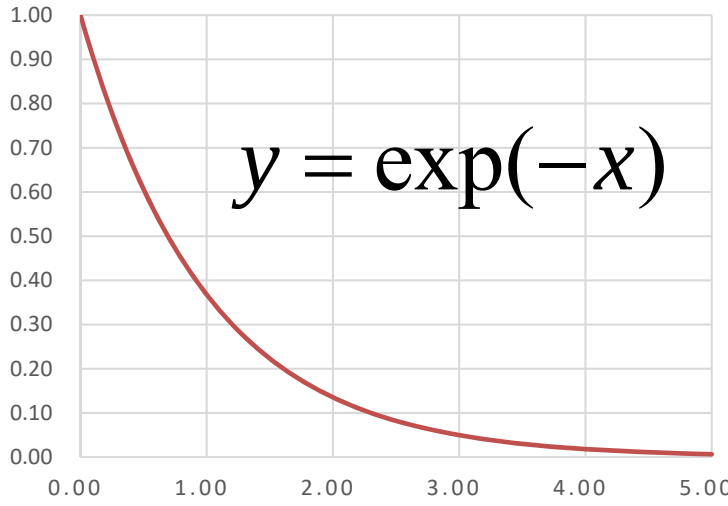
# 基本的なアイディア basic idea

確率 $exp(-\Delta E/T(t))$ で新しい解を採択（新しい解が優れていなくても
ある一定の確率で採択する）

Take new solution (B) with probability $exp(-\Delta E/T(t))$ even if (B) is worse than (A)

$\Delta E / T(t)$ 反復回数が増えるにつれ大きくなる
Increases as number of iterations increases
新しい解の目的関数値が大きいと大きくなる
Increases as objective function of new solution is larger

悪い解の採択確率
Possibility of taking worse solution




反復回数が増えるにつれ，悪い解の
採択確率が減少
Possibility of taking worse solution decreases as number of iterations increases
新しい解の目的関数値が大きいと
悪い解の採択率が減少
Possibility of taking worse solution decreases as objective function of new design is worse

# 演習 exercise

確率 $exp(-\Delta E/T(t))$で新しい解を採択（新しい解が優れていなくてもある一定の確率で採択する）

*Take new solution (B) with probability $exp(-\Delta E/T(t))$ even if (B) is worse than (A)*

$y=exp(-\Delta E/T(t))$を図示してみよう

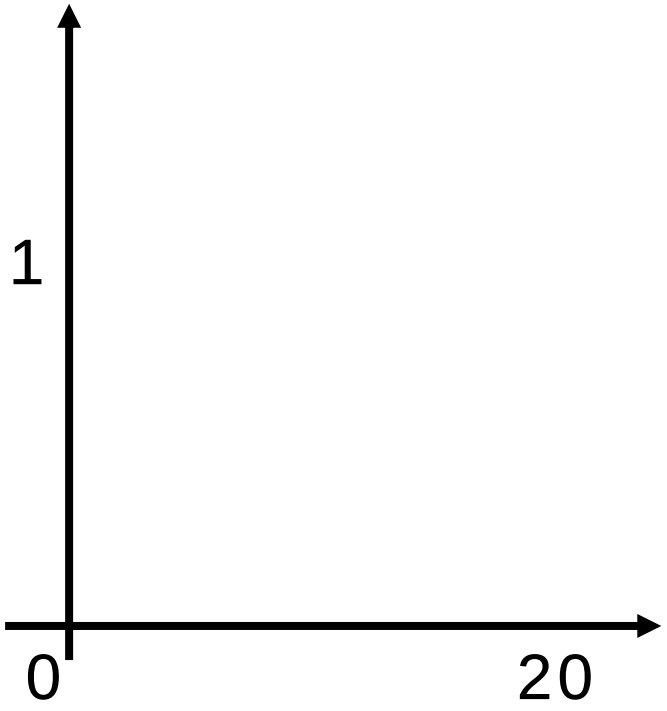
draw $y=exp(-\Delta E/T(t))$

$$\Delta E = E_{new} - E_{current} = 0.1$$

$$T(t) = \alpha^t T_0 \quad \alpha = 0.8$$

$$T_0 = 1$$

$$t = 1, 2, 3, \ldots, 20$$



# 演習 exercise

確率 $exp(-\Delta E/T(t))$で新しい解を採択（新しい解が優れていなくてもある一定の確率で採択する）

*Take new solution (B) with probability $exp(-\Delta E/T(t))$ even if (B) is worse than (A)*

$y=exp(-\Delta E/T(t))$を図示してみよう

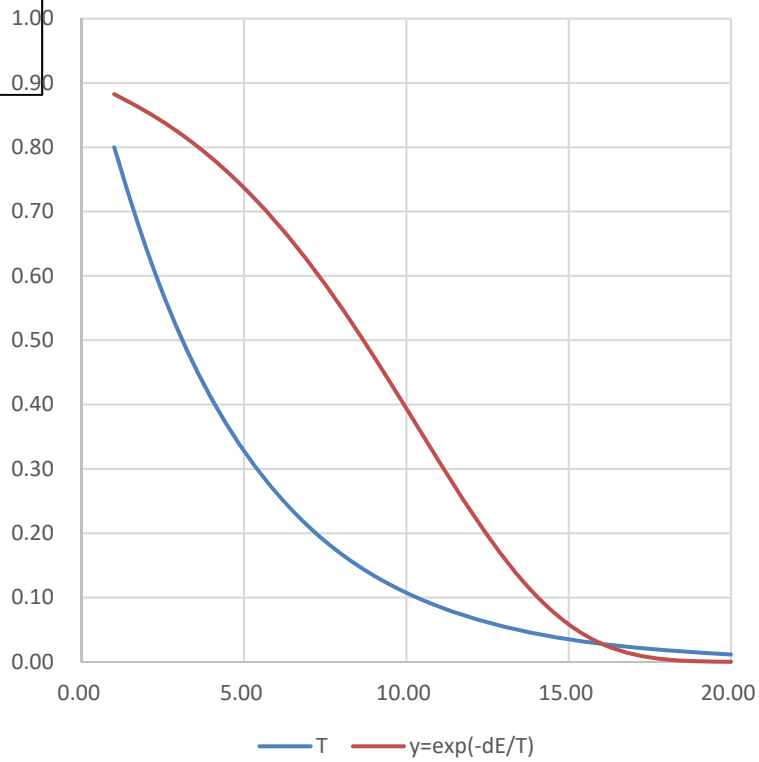
draw $y=exp(-\Delta E/T(t))$

$$\Delta E = E_{new} - E_{current} = 0.1$$

$$T(t) = \alpha^t T_0 \quad \alpha = 0.8$$

$$T_0 = 1$$

$$t = 1, 2, 3, \ldots, 20$$



# 計算例 example

次の関数の最小値を焼き鈍し法を使って求めよ.
Obtain minimum solution using SA

$$f(x)=x^4-16x^3+88x^2-190x+144$$

*(1) ある解(A)の近傍に新しい解(B)をランダムに作成し，目的関数値を評価する.*

*generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)*

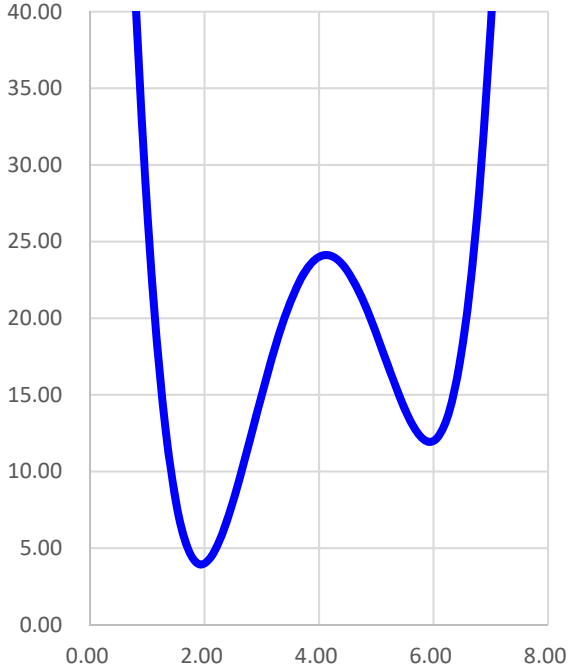
*(2) もとの解と新しい解の目的関数値の差$\Delta E$が負であれば(新しい設計が優れていれば)新しい解を採択*

*If objective function difference $\Delta E$ is negative (if (B) is better) , take new solution (B)*

*(3) そうでなければ，確率 $\exp(-\Delta E/T(t))$ で新しい解を採択(新しい解が優れていなくてもある一定の確率で採択する)*

*Else take new solution (B) with probability of*

*$\exp(-\Delta E/T(t))$*



# Excel sheet SA(1)

(1) ある解（A)の近傍に新しい解（B)をランダムに作成し，目的関数値を評価する.

generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)

(2) もとの解と新しい解の目的関数値の差$\Delta E$が負であれば（新しい設計が優れていれば）新しい解を採択

If objective function difference $\Delta E$ is negative (if (B) is better) , take new solution (B)

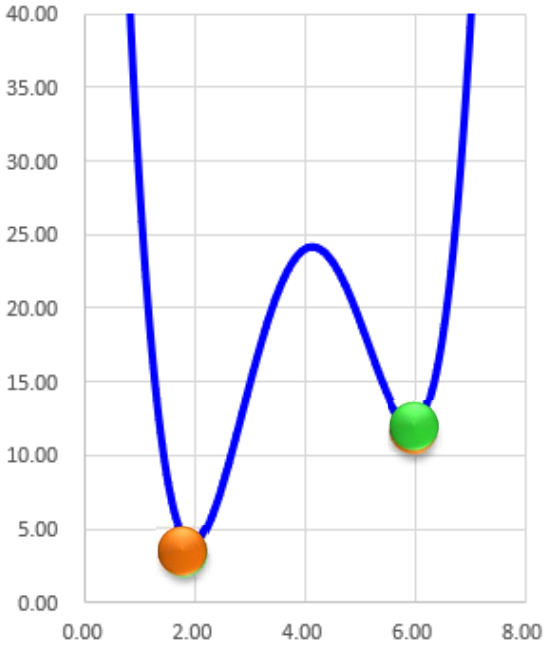
(3) そうでなければ，確率 $exp(-\Delta E/T(t))$で新しい解を採択（新しい解が優れていなくてもある一定の確率で採択する）

Else take new solution (B) with probability of $exp(-\Delta E/T(t))$

$$-0.1<\Delta x<0.1$$

$$\exp\left(-\Delta E/(0.95^{t} \times 1.0)\right)$$

| initial solution | optimized solution |
|---|---|
| x=1 | 1.94 |
| x=2 | 1.94 |
| 3 | 1.94 |
| 4 | 1.94 |
| 5 | 5.93 |
| 6 | 5.93 |
| 7 | 5.93 |
| 8 | 5.93 |
| 9 | 5.93 |
| 10 | 5.93 |



# アルゴリズム algorithm

(1) 初期解を定める　　define initial solution

(2) 時刻をゼロとする.　　set time is zero　t=0

(3) ある解（A)の近傍に新しい解（B)をランダムに作成し，目的関数値を評価する.

generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)

(4) もとの解と新しい解の目的関数値の差$\Delta E$が負であれば（新しい設計が優れていれば）新しい解を採択

If objective function difference $\Delta E$ is negative (if (B) is better) , take new solution (B)

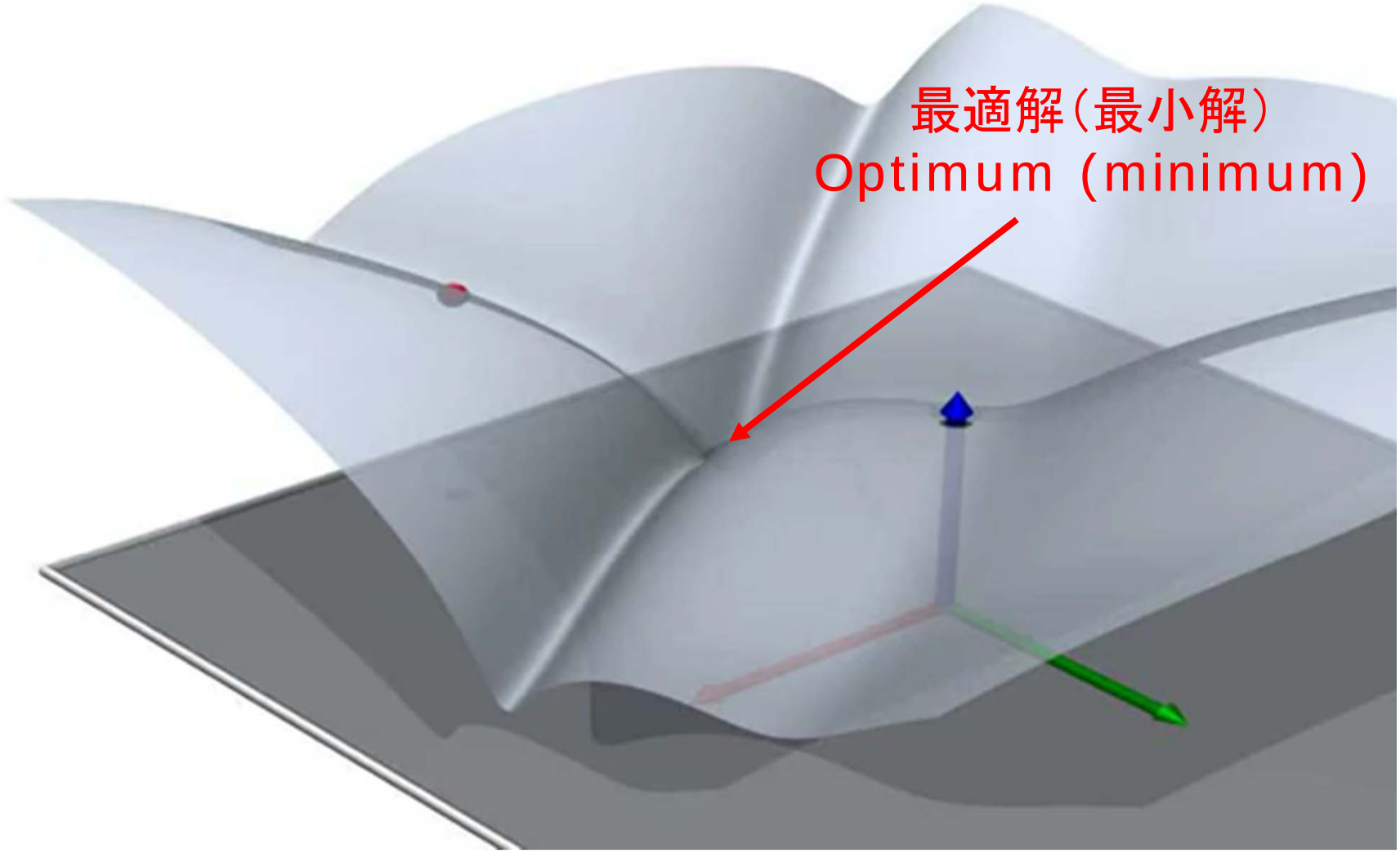
(5) そうでなければ，確率 $exp(-\Delta E/T(t))$で新しい解を採択（新しい解が優れていなくてもある一定の確率で採択する）

Else take new solution (B) with probability of $exp(-\Delta E/T(t))$

(6) 時刻をすすめる　　time advances　t=t+1

(7) ステップ(3) にもどる　　go to step (3)

# 焼き鈍し法の実行例 visualization of SA




https://www.youtube.com/watch?v=iaq_Fpr4KZc

色は温度を表す color shows tempurature

# 演習　次の関数の最小値を焼き鈍し法で求めてみよう

exercise　Obtain minimum using SA

$$f(x) = x_1^2 - 16x_1 + 2x_2^2 - 16x_2$$

初期値 initial solution $(x_1,x_2)=(2,1)$

*(1) 初期解を定める　　define initial solution*

*(2) 時刻をゼロとする.　　set time is zero　　t=1*

*(3) ある解(A)の近傍に新しい解(B)をランダムに作成し，目的関数値を評価する.*

*generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)*

*(4) もとの解と新しい解の目的関数値の差$\Delta E$が負であれば(新しい設計が優れていれば)新しい解を採択*

*If objective function difference $\Delta E$ is negative (if (B) is better) , take new solution (B)*

*(5) そうでなければ，確率 $\exp(-\Delta E/T(t))$ で新しい解を採択(新しい解が優れていなくてもある一定の確率で採択する)*

*Else take new solution (B) with probability of $\exp(-\Delta E/T(t))$*

*(6) 時刻をすすめる　　time advances　　t=t+1*

*(7) ステップ(3) にもどる　　go to step (3)*

$$f(x) = x_1^2 - 16x_1 + 2x_2^2 - 16x_2$$

初期値 initial solution $(x_1, x_2)=(2,1)$

*(3) ある解(A)の近傍に新しい解(B)をランダムに作成し，目的関数値を評価*

*generate solution (B) near solution (A) randomly. Then evaluate objective function of (B)*

*(4) もとの解と新しい解の目的関数値の差ΔEが負であれば(新しい設計が優れていれば)新しい解を採択*

*If objective function difference ΔE is negative (if (B) is better) , take new solution (B)*

*(5) そうでなければ，確率 exp(-ΔE/T(t)) で新しい解を採択(新しい解が優れていなくてもある一定の確率で採択する)*

*Else take new solution (B) with probability of exp(-ΔE/T(t))*

$$x_{new,i} = x_i + a_i \cdot \Delta x_i$$

$$a_i = (\text{rand} - 0.5) \times 2$$

$$0 \le \text{rand} \le 1$$

$$\Delta x_i = 0.1$$

if $\Delta E = E_{new} - E_{current} < 0$

$$\vec{x} = \vec{x}_{new}$$

if $\Delta E = E_{new} - E_{current} \ge 0$

and $rand \le \exp(-\Delta E / T(t))$

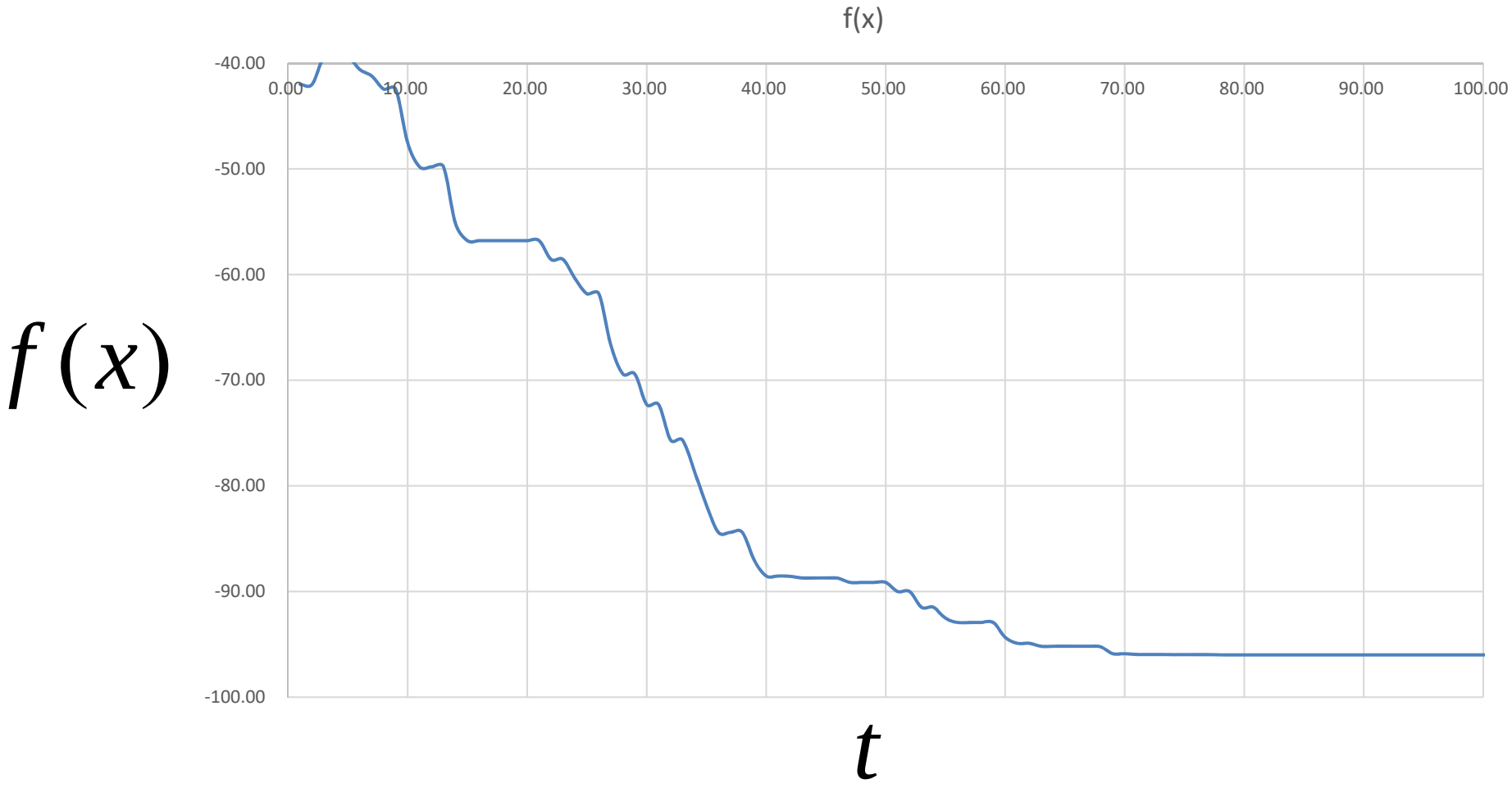
$$\vec{x} = \vec{x}_{new}$$

$$T(t) = \alpha^t T_0 \qquad t = 1, 2$$

$$\alpha = 0.9 \quad T_0 = 1$$

$$f(x) = x_1^2 - 16x_1 + 2x_2^2 - 16x_2$$

初期値 initial solution $(x_1,x_2)=(2,1)$



# 焼き鈍し法の利点と欠点は？

What's the advantages/disadvantages of SA?

ちなみに，最急降下法の利点と欠点は

Reference: advantages/disadvantages of steepest descending method

- 利点：勾配情報を用いているので探査効率が良い

  Advantage: high efficiency because gradient information is used

- 欠点：多峰的な問題では局所解しか見つけられない

  Disadvantage: local optima when multimodal

- 欠点：多目的最適化問題に用いる場合は多数あるパレート最適解のうちの１つしか見つけることができない

  Disadvantage: only one of Pareto-optimal designs is found

- 欠点：不連続最適化問題では計算ができない

  Disadvantage: not applicable to objective function with discontinuity

# 利点・欠点 Advantage/disadvantage of SA

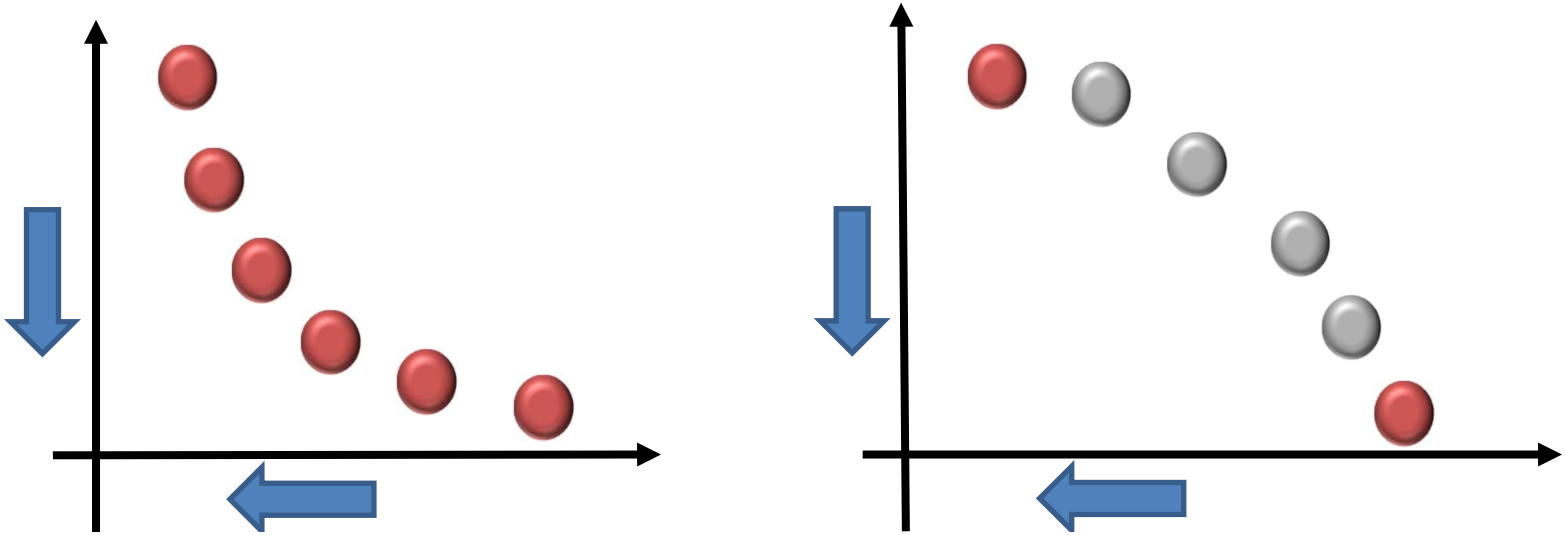
- ある割合で改悪される方向への移動を許すので，局所解に陥りにくい

  Possibility of global search

- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない

  Find only of the Pareto-optimal solutions

$$f = a \cdot f_1 + (1-a) \cdot f_2$$



# 利点・欠点 Advantage/disadvantage of SA

- ある割合で改悪される方向への移動を許すので，局所解に陥りにくい

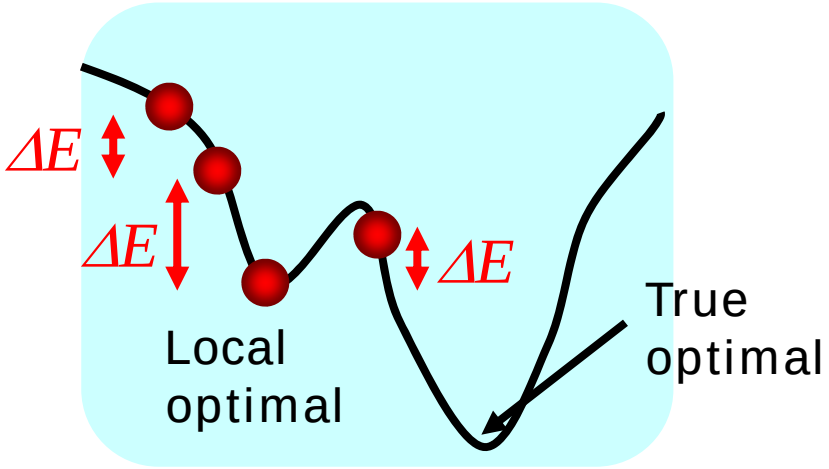
  Possibility of global search

- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない

  Find only of the Pareto-optimal solutions

- 不連続最適化問題も解くことが可能

  Applicable to design problems with discontinuous objective function



# 利点・欠点 Advantage/disadvantage of SA

- ある割合で改悪される方向への移動を許すので，局所解に陥りにくい

  Possibility of global search

- 多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない

  Find only of the Pareto-optimal solutions

- 不連続最適化問題も解くことが可能

  Applicable to design problems with discontinuous objective function

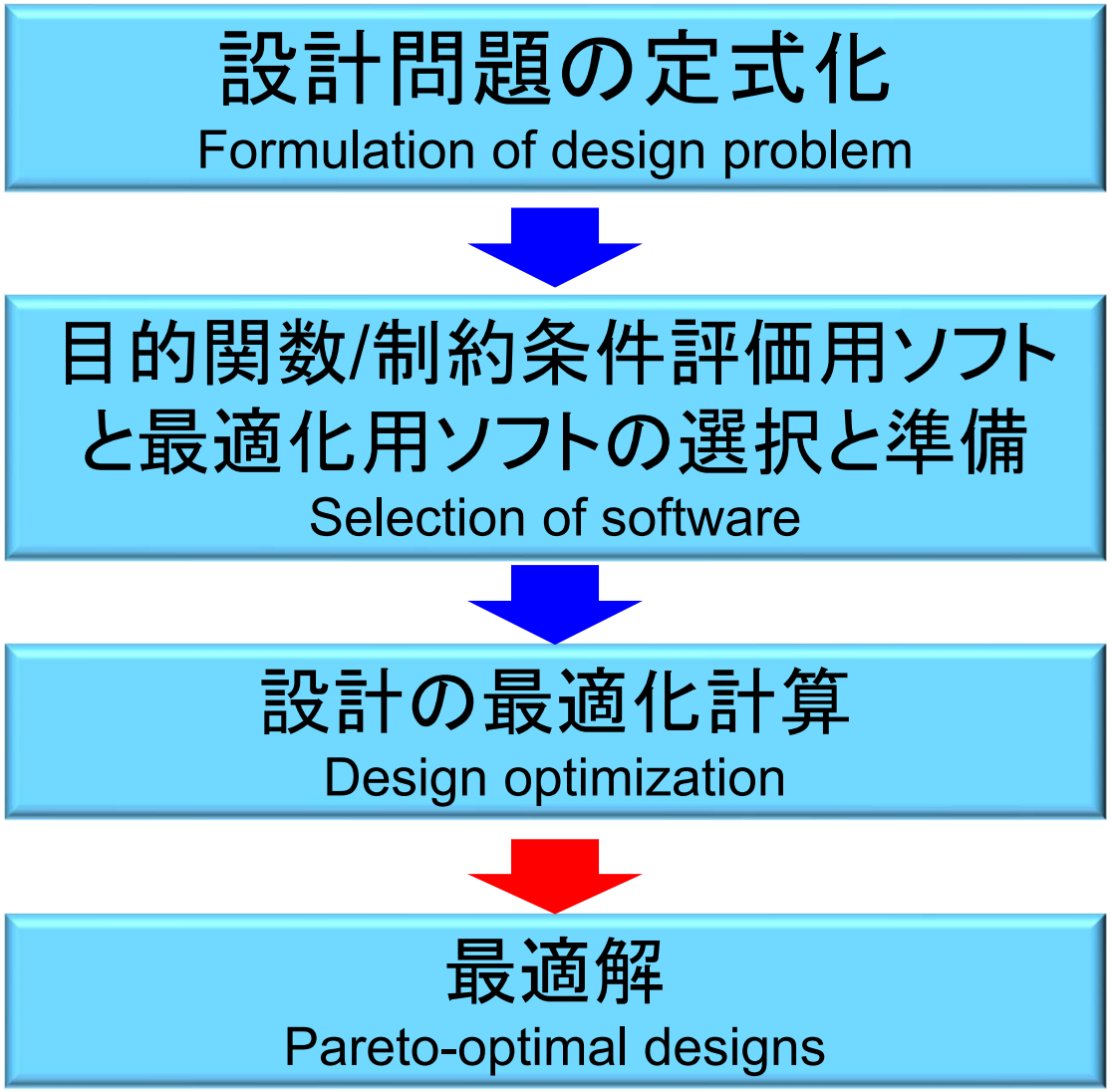
- 勾配情報は用いないので探索効率は最急降下法よりも劣る

  Less efficient because gradient is not used

# まとめ

summary

# 設計最適化の手順 process of optimization



# 設計最適化手法　design optimization methods

- 最急降下法 (steepest descending method)
  - 利点：勾配情報を用いているので探査効率が良い

    Advantage: high efficiency because gradient information is used
  - 欠点：多峰的な問題では局所解しか見つけられない

    Disadvantage: local optima when multimodal
  - 欠点：多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない

    Disadvantage: only one of Pareto-optimal designs is found
  - 欠点：不連続最適化問題では計算ができない

    Disadvantage: not applicable to objective function with discontinuity

# 設計最適化手法　design optimization methods

- 焼き鈍し法 (simulated annealing)
  - 利点：多峰的な問題でも大域的な最適解を見つけることが可能

    Advantage: global optimum even when multimodal
  - 利点：不連続最適化問題でも最適解を見つけることが可能

    Advantage: applicable to discontinuous objective function
  - 欠点：探査効率で最急降下法に劣る

    Disadvantage: less efficiency compared with gradient-based methods
  - 欠点：多目的最適化問題に用いる場合は多数あるパレート最適解のうちの1つしか見つけることができない

    Disadvantage: only one of Pareto-optimal solutions is found.